From Traces to Guarded Programs: Evidence-Gated Compilation of Recurrent Agent Workflows

Anonymous authors
Paper under double-blind review

Abstract

Tool-using agents often invoke a model at every step, even along recurrent read-only paths. Replacing repeated reasoning with deterministic execution could reduce inference cost, but recurrence alone does not make substitution safe: tool arguments may depend on prior observations, apparently read-only operations may hide effects, and replaying the same calls may still alter the final answer. Recurrence identifies an optimization opportunity, not permission to remove reasoning.

We introduce *guarded agentic compaction* (GAC), an evidence-gated approach that compiles recurrent execution paths into deterministic programs only when substitution is justified. GAC compiles initial read-only prefixes only: it grounds tool arguments in observable state, treats writes, approvals, handoffs, and unknown effects as hard barriers, protects compiled execution with input, output, and environment guards, and admits specialization only when finite-sample evidence satisfies configured risk and confidence requirements. Otherwise, it retains the largest justified prefix when possible or falls back to the unchanged agent. Admission certificates are exact finite-sample bounds under i.i.d. calibration groups, compiler-wide where one candidate reached calibration and per-candidate where two did (two of three primary families, whose corrected bound is 0.057).

Across three live GitHub workflow families and 90 unseen test cases, GAC satisfies 90/90 exact task contracts and the unchanged agent 89/90, a difference of one stochastic baseline miss, while reducing model requests by 66.6%, tokens by 63.1%, latency by 64.2%, and estimated cost by 58.7%. A time-forward evaluation across five frozen public repositories specializes four and refuses one. On four public trace benchmarks, NESTFUL, API-Bank, and BFCL v4 contain recurrent, replayable traces but insufficient qualifying evidence to satisfy GAC's default 0.05 risk bound at the required confidence, whereas AppWorld provides sufficient evidence for admission. On a second model family and a second provider, the unchanged pipeline re-derives the same artifacts from each model's own traces on two of the three families and refuses the third.

GAC reframes agent optimization as evidence-gated specialization: compile only what is justified, dispatch only when runtime conditions remain valid, and preserve model reasoning everywhere else.

1 Introduction

The standard agent loop pays for a model decision at every tool boundary (Yao et al., 2023). That flexibility is valuable while a workflow is novel, but mature agents often revisit the same read-only evidence path with the same data dependencies, where repeating the decision adds latency, tokens, and cost without adding adaptation.

Removing that decision is not ordinary caching. A tool argument may depend on a specific earlier observation, a repeated sequence may cross an approval or write, an apparently read-only tool may hide an undeclared effect, and replaying the same outputs may still change the downstream answer. The question is when a recurrent model-mediated region may be replaced by a deterministic program, and when it must be refused.

Consider held-out issue #4420. Given only `issue_number=4420`, the observed trace calls `record(4420)`, passes the *returned* `record.issue_number` to `labels` and `comments`, and in-

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

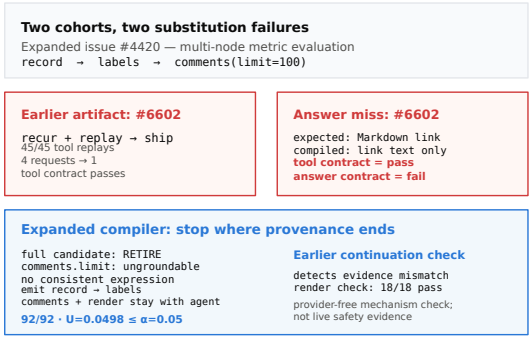


Figure 1: Two retained cohorts expose different failure modes: issue #4420 forces a shorter grounded prefix, and issue #6602 shows that clean tool replay need not preserve the downstream answer.

vokes the latter with `limit=100`. All 132 discovery executions take that route, so a recurrence-only optimizer would emit all three reads. GAC instead retires the *region*: `comments.limit` is reachable neither from the entry state nor from either earlier result, so one unwitnessed slot blocks the three-read candidate as `ungroundable_slot`. What survives is the maximal justified prefix `record` → `labels`; the agent still chooses the limit and writes the answer. That two-read artifact passes 92/92 calibration groups at $\alpha = 0.05$ (upper bound 0.0498).

An earlier 18-record study supplies the counterexample: its three-read artifact replayed all 45 tool calls, yet issue #6602 lost the URL from a Markdown link and only 17/18 downstream answers stayed exact; a provider-free continuation guard catches that miss. Together the two cohorts (fig. 1) show why recurrence only proposes a candidate.

The closest precedent is the tracing just-in-time compiler: Dynamo (Bala et al., 2000) detects a hot path, compiles it behind a *guard*, and deoptimizes when the guard fails (Hölzle et al., 1992). Agent traces admit the same structure — hot-trace detection becomes family mining, guard synthesis an entry contract, deoptimization fallback to the unchanged agent — but add three obligations, and claim one capability less: a JIT enters a trace at an arbitrary program point, whereas GAC compiles only *initial* read-only prefixes (eq. (2)), because its runtime resolves artifacts at the entry boundary. The obligations: tool arguments must be shown *grounded* in observable state; declared *effects* must permit speculation, since an agent's "interpreter" calls the outside world; and the residual error rate, being statistical, needs a finite-sample bound, not a soundness proof.

We study these obligations through *guarded agentic compaction* (GAC), a compile-or-retire trace-to-program system for recurrent read-only prefixes (fig. 2). The core contribution is not "agent compilation" in general, but an admissibility argument for trace-derived specialization:

1. A guarded specialization pipeline (Alg. 1) combining typed provenance, effect and position barriers, bounded synthesis, runtime verification, and finite-sample compile-or-retire admission (Alg. 2, Proposition 1) with the certificate level stated per artifact — to our knowledge the first exact finite-sample admission certificate for trace-derived agent compilation; every stage can refuse, and the default output is the unchanged agent.
2. A real-provider evaluation across three public-record workflow families, a narrower cross-repository, time-forward extension keeping both artifacts and principled retirements, and same-cohort replications on a second model family and a second provider.
3. Calibrated refusal: recurrence and replayability do not license deployment. NESTFUL, API-Bank, and executed BFCL gold plans clear provenance, synthesis, and replay with no wrong execution and retire exactly where their support falls below the exact sample floor; the discovered programs match hand-written ones wherever the workflow is already known.
4. A pre-registered fourth substrate, AppWorld — which admits — plus a measurement on 8,190 released baseline trajectories where the admitted region is eligible on 2,339/2,340 full-code but only 1/4,680 ReAct or plan-and-execute trajectories, separating *admissible* from *dispatchable*.

2 Problem Formulation

Episodes and candidate regions. An episode $E = (z, M, e_{1:T}, y)$ contains an entry-state snapshot z, a content-addressed execution manifest M, ordered events e_t (model requests and responses, tool calls and results, handoffs, guardrails, approvals, errors, commit boundaries), and an observable outcome y; c_j denotes the j-th tool call and o_j its result. Each tool f carries an application-owned effect declaration $\epsilon(f)$ and capability flags such as *speculatable*; an unknown effect is not a read.

A candidate region $R = [a, b]$ begins at a model-request boundary and ends after a tool result. It is *groundable* when every call argument is either an application-declared literal for that slot or an expression over entry state or prior results inside R,

$$\forall c_j \in R,\ \forall u \in \mathrm{args}(c_j): \quad \mathrm{Lit}(c_j, u) \ \vee\ \exists\, s \in \mathrm{Src}(z, o_{<j}),\ \exists\, g_{j,u} \in \mathcal{L}:\ u = g_{j,u}(s), \tag{1}$$

where Lit is a schema-level allowlist fixed independently of the traces, $\mathcal{L}$ is a closed, bounded transform library, and $\mathrm{Src}(z, o_{<j})$ enumerates the flattened paths of z and of each prior result o_i, $i < j$; and *effect-admissible* when every tool in R is declared read-only, speculatable, replayable, and inside one principal and isolation partition. The witness is deliberately *single-source* — one chain over one observation, not a join over the history — which is the quantifier Alg. 3 implements. Handoffs, approvals, writes, errors, and unknown effects are hard barriers. Because the deployed runtime resolves artifacts at the initial model boundary, the compiler also imposes a *position invariant*:

$$a = 0 \quad \text{in the normalized model-boundary index.} \tag{2}$$

A suffix program dispatched at entry can reorder or duplicate calls the agent has already made; a pilot that violated eq. (2) produced exactly that fault (appendix E). Everything that follows is therefore an *initial read-only prefix* compiler; section 5.3 prices what that costs.

Selective optimization objective. The compiler emits an artifact $A = (P, H, V, q, \eta, M)$: a deterministic program, hard guard, output verifier, nonconformity score, threshold, and manifest pins. Let $x = (z, M', \omega)$ be a future entry state, runtime manifest, and external state, where $M' \simeq M$ is equality on every pinned compatibility field and ω affects outcomes and cost but is unobservable at the dispatch boundary. Dispatch is *selective*,

$$d_A(x) = \mathbf{1}\{\, H(z, M') = 1 \ \wedge\ q(z) \le \eta \ \wedge\ M' \simeq M \,\}, \tag{3}$$

and any failed precondition runs the unchanged baseline B. If execution fails before a commit and staging proves the attempt clean, B runs; a dirty failure after a commit is an incident, not a fallback. Fix $L(A, x) = 1$ for an externally visible task/contract violation or incident after dispatch, $L = 0$ for a clean abort followed by unchanged B, and let $C(A, x)$ be the complete realized cost of the attempt including a clean fallback. For a future group $G = (x_1, \ldots, x_m)$, set $D_A(G) = \max_i d_A(x_i)$ and $W_A(G) = \max_i d_A(x_i) L(A, x_i)$. We seek

$$\max_A\ \mathbb{E}_{G \sim \mathcal{D}_{\mathrm{grp}}}\Big[\, \textstyle\sum_{x \in G} d_A(x)\,\{C(B, x) - C(A, x)\}\,\Big] \quad \text{s.t.} \quad \Pr_{G \sim \mathcal{D}_{\mathrm{grp}}}[W_A(G) = 1 \mid D_A(G) = 1] \le \alpha. \tag{4}$$

Both halves take the same group measure. *First*, the constraint conditions on dispatch: abstention does not enter its denominator, although it earns no savings in the objective; by convention its conditional risk is zero when population dispatch probability is zero. *Second*, returning *no* artifact is feasible, so RETIRE is a legitimate output, and eq. (4) is a design target rather than a solved program — the implementation calibrates a bounded ranked set and retains nondominated survivors, and we prove neither optimality nor any guarantee for that search beyond the per-candidate and fixed-m cases of section 3. *Third*, L scores the *episode*: a sound prefix whose baseline continuation errs still contributes $L = 1$, so the gate certifies end-to-end compliance under substitution, not non-inferiority, and admission requires B to already succeed on a $1 - \alpha$ share of the dispatched slice (appendix G).

3 Guarded Agentic Compaction

Figure 2 and Alg. 1 give the same six stages at two levels of detail. The order is part of the argument: each stage can only weaken a candidate, so a later stage may reject one an earlier stage passed, but never authorize one an earlier stage blocked.

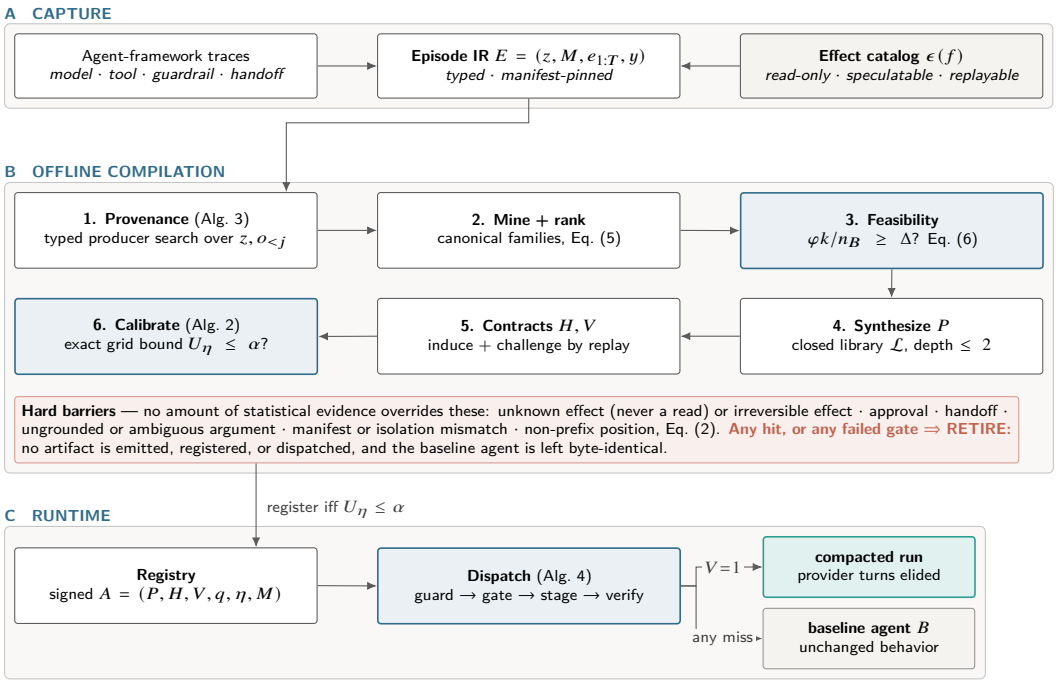


Figure 2: Architecture of GAC. **(A)** Capture normalizes traces into one typed Episode IR against a declared effect catalog. **(B)** Compilation is a cascade of independent rejection points; the shaded band lists barriers no statistical evidence can override. **(C)** At runtime one terminal edge compacts; the rest return the unchanged agent.

Algorithm 1 COMPILE — the compile-or-retire cascade. Every candidate is either registered or assigned an attributed RETIRE, so refusal accounting remains explicit. Stages marked (CALLER) are separate library entry points run in this order rather than steps inside one function.

Input: episodes $\mathcal{E}$; effect catalog C; entry schema Θ; transform library $\mathcal{L}$; ambiguity cap κ; mining window $w_{\max}$; budgets α, δ; grid Λ; feasibility target Δ
Output: artifact set $\mathcal{A}$, or RETIRE with attributed reasons

$\mathcal{A} \leftarrow \emptyset$
$\mathcal{E} \leftarrow \text{QUALIFY}(\mathcal{E}, C)$; $\{\mathcal{E}_M\} \leftarrow \text{PARTITIONBY}(\mathcal{E}, \text{manifest, principal, isolation})$ ▷ drop partial runs, unpinned manifests
for all partitions $\mathcal{E}_M$ **do**
  $(\mathcal{T}, \mathcal{D}, \mathcal{K}, \mathcal{S}) \leftarrow \text{GROUPEDSPLIT}(\mathcal{E}_M)$ (CALLER) ▷ train / dev / calibration / sealed test, split by *group*
  $(G, S) \leftarrow \text{BUILDPATG}(\mathcal{T}, \Theta, C, \mathcal{L}, \kappa)$ ▷ Alg. 3: annotate every slot
  $\mathcal{F} \leftarrow \text{CANONICALIZE}(\text{MINEREGIONS}(G, S, C, w_{\max}))$, keeping only cross-group support ▷ rejects any window with an UNGROUNDED or AMBIGUOUS slot
  if $\text{CEILING}(\mathcal{T}, \mathcal{F}) < \Delta$ **then** (CALLER)
    record RETIRE **(infeasible ceiling, Eq. (6))**; **continue** ▷ try the next compatible partition
  end if
  for all families $F \in \mathcal{F}$ ranked by Eq. (5) **do**
    $P \leftarrow \text{SYNTHESIZE}(F, \mathcal{L})$ at depth ≤ 2; **if** $P = \perp$ **then record** RETIRE **(synthesis); continue**
    $(H, V) \leftarrow \text{INDUCECONTRACT}(F)$; **if** $\neg\text{REPLAYANDCHALLENGE}(P, V, \mathcal{D})$ **then record** RETIRE **(counterexample); continue**
    $q \leftarrow \text{FITSCORE}(\mathcal{D})$; **freeze** q ▷ dev groups only; never calibration
    $\eta \leftarrow \text{CALIBRATE}(P, H, V, q, M, \mathcal{K}, \Lambda, \alpha, \delta)$; **if** $\eta = \perp$ **then record** RETIRE **(risk); continue** ▷ Alg. 2
    $\mathcal{A} \leftarrow \mathcal{A} \cup \{\text{PACKAGE}(P, H, V, q, \eta, M)\}$ with evidence and lifecycle
  end for
end for
return $\text{SELECTUNDOMINATED}(\mathcal{A})$ if nonempty, else RETIRE

1. Typed provenance. A capture adapter normalizes framework traces into the Episode IR; manifests pin prompt, policy, guardrail, tool, model, SDK, tracer, entry-contract, and effect-catalog identities, and episodes with different compatibility keys are partitioned before any learning. For each call-argument slot, the program-argument trace graph (PATG, Alg. 3) searches entry-state and prior-result paths for typed values reachable by a bounded transform (database-style provenance (Cheney et al., 2009) for tool arguments). Two slot marks make the region miner block rather than guess: a slot with *no* witness is a genuine model decision, and one with more than $\kappa = 3$ witnesses is ambigu-

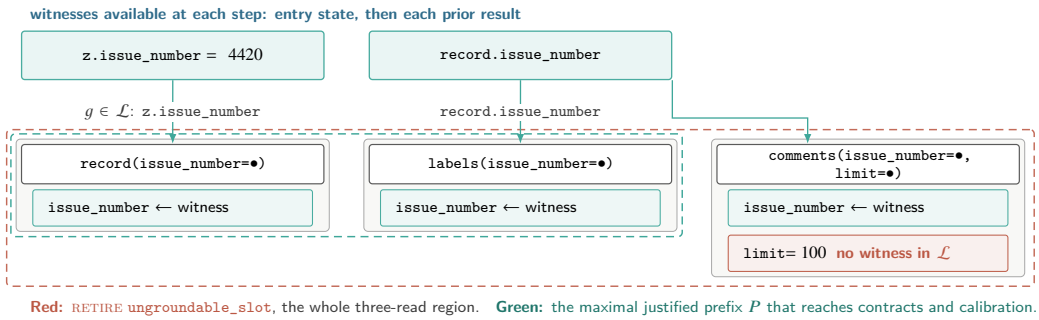


Figure 3: Equation (1) on the held-out issue-4420 record. Every argument slot needs an expression in $\mathcal{L}$ over the entry state or a prior result; the two `issue_number` slots have one and the `limit` slot has none. The question is strictly harder than recurrence — not "did this sequence repeat?", which all 132 discovery executions did, but "can every socket be wired from an allowed prior value?" One unwitnessed slot retires the whole region, and so does a slot with more than κ witnesses.

ous (fig. 3). A repeated tool name, a matching literal, and a successful replay are not witnesses; only an expression in $\mathcal{L}$ is.

2. Region mining and ranking. The miner enumerates model-boundary-to-result windows of at most $w_{\max}$ tool calls, qualifies live-ins and effects against a human-signed rather than inferred catalog, and hashes tool signatures, dependency topology, run shape, and live-in/live-out shape while abstracting literals. Families require support across independent scenario *groups*, not raw event frequency. Survivors are ranked by an MDL-inspired trade-off that pays for expected savings and charges for heterogeneity, effect exposure, and program size:

$$\text{score}(F) = \underbrace{s_F\,\bar{k}_F\,c_m}_{\text{support-weighted saving}} - \lambda_1 \mathsf{Ent}(F) - \lambda_2 \rho_{\text{eff}}(F) - \lambda_3 |F|, \tag{5}$$

with s_F supporting train groups, $\bar{k}_F$ the mean removable model requests, c_m their unit cost, and $\mathsf{Ent}(F)$ the Shannon entropy over canonical variants inside the family. Equation (5) affects only *which* family is tried first.

3. Feasibility ceiling. Before any synthesis runs, an estimator bounds what the compiler could achieve with a perfect verifier and a gate that never abstains:

$$\Delta_{\max} = \frac{\varphi\, k}{n_B} \;\geq\; \Delta \quad \text{(pre-synthesis filter)}, \qquad \varphi\,\rho\,k \;\geq\; \Delta\, n_B \quad \text{(realized, post-verification)}, \tag{6}$$

with n_B baseline model requests per episode, φ the fraction of episodes holding an eligible region, k the requests removed per successful dispatch, and ρ the verifier pass rate. Stage 3 runs before synthesis, so it evaluates only the left form ($\rho = 1$); the right form is what the workload must meet once V is induced, and since $\rho \leq 1$ no measured reduction exceeds $\Delta_{\max}$, so a workload under target is declined before a compiler is built for it.

4–5. Bounded synthesis and contracts. Synthesis searches the 23-operator library of table 5 at depth at most two; bindings are ranked by cross-group stability, and a group-wise refit rejects bindings exploiting row-level coincidence. The hypothesis space is small enough to read, in the spirit of programming-by-example (Gulwani, 2011) and bounded sketching (Solar-Lezama et al., 2006), and never generates arbitrary Python. Hard guards then constrain entry types, required fields, categorical sets, numeric intervals, patterns, isolation keys, allowed effects, and manifest pins; verifiers constrain live-out presence, type, cardinality, empirical hulls, provenance, effect multisets, and call counts. These are candidate invariants, not proofs — which is why a statistical gate follows rather than replaces them.

Algorithm 2 CALIBRATE — fixed-grid exact selective admission. The score q and the grid Λ are frozen *before* this procedure sees a calibration group, which is what makes the union bound legitimate.

Input: frozen candidate $A_0 = (P, H, V, q, M)$ without threshold; calibration groups $\mathcal{K}$ with fixed per-instance violations $L(A_0, x)$; grid Λ; budgets α, δ
Output: threshold η with a simultaneous guarantee, or $\perp$ (RETIRE)

$\Pi \leftarrow \emptyset$ ▷ admissible (coverage, threshold) pairs, distinct from Alg. 1's $\mathcal{A}$
for all $\eta \in \Lambda$ **do**
  define $d^\eta(x)$ by Eq. (3) with threshold η; $D^\eta(g) \leftarrow \max_{x \in g} d^\eta(x)$; $W^\eta(g) \leftarrow \max_{x \in g} d^\eta(x) L(A_0, x)$
  $K_\eta \leftarrow \{ g \in \mathcal{K} : D^\eta(g) = 1 \}$; $n_\eta \leftarrow |K_\eta|$; **if** $n_\eta = 0$ **then continue** ▷ vacuous bound $U_\eta = 1$
  $k_\eta \leftarrow |\{ g \in K_\eta : W^\eta(g) = 1 \}|$ ▷ *wrong after dispatch* only, never abstention
  $U_\eta \leftarrow 1$ if $k_\eta = n_\eta$, else $\mathrm{Beta}^{-1}\big(1 - \frac{\delta}{|\Lambda|}; k_\eta + 1, n_\eta - k_\eta\big)$; **if** $U_\eta \le \alpha$ **then** $\Pi \leftarrow \Pi \cup \{(n_\eta/|\mathcal{K}|, \eta)\}$ ▷ Eq. (7)
end for
return $\perp$ if $\Pi = \emptyset$, else the η component of $\mathrm{lexmax}_{(\mathrm{cov}, \eta) \in \Pi}(\mathrm{cov}, \eta)$ ▷ largest empirical group coverage; larger threshold breaks ties

6. Exact risk-gated admission. Algorithm 2 decides deployment. The score q is a logistic model over entry-observable features — missing fields, hull margin, temporal drift, and provenance ambiguity — fitted by regularized maximum likelihood on *dev* groups only, never on calibration groups, then frozen together with the finite threshold grid $\Lambda = \{0.02, 0.05, 0.08, 0.11, 0.14, 0.17, 0.20, 0.25, 0.30, 0.40, 0.50\}$. q is *fitted* to predict an *unproductive* dev group — guard rejection, abstention, or manifest mismatch — never a wrong execution, since REPLAYANDCHALLENGE already retired those; the bound below is *counted* only on dispatched groups that turned out wrong (appendix C). For each $\eta \in \Lambda$ the compiler counts admitted calibration groups (n_η) and those containing at least one post-dispatch violation (k_η), and inverts the one-sided exact binomial test (Clopper & Pearson, 1934) at a Bonferroni-corrected level, with explicit empty and all-violation cases:

$$U_\eta = \begin{cases} 1, & n_\eta = 0 \text{ or } k_\eta = n_\eta, \\ \mathrm{Beta}^{-1}\Big(1 - \frac{\delta}{|\Lambda|}; k_\eta + 1, n_\eta - k_\eta\Big), & 0 \le k_\eta < n_\eta, \end{cases} \qquad U_\eta\big|_{k_\eta = 0} = 1 - \Big(\frac{\delta}{|\Lambda|}\Big)^{1/n_\eta}. \quad (7)$$

It admits the largest-coverage threshold with $U_\eta \le \alpha$ and retires otherwise — a fixed-grid instance of learn-then-test (Angelopoulos et al., 2022). The closed form on the right is the operative constraint here: at $\alpha = 0.05$, $\delta = 0.1$, $|\Lambda| = 11$, a zero-violation family still needs $n_\eta \ge 92$ admitted groups, and the default output is refusal.

That floor also bounds what selectivity is *certifiable*: coverage $c = n_\eta/|\mathcal{K}|$ needs $|\mathcal{K}| \ge \lceil 92/c \rceil$, so on the $|\mathcal{K}| = 92$ cohorts used throughout — the smallest satisfiable pool — the only admissible coverages are $c \in \{0, 1\}$ whatever q ranks, since abstaining on one group leaves $U_\eta = 0.0503 > \alpha$. Intermediate coverage can still be *observed*; it cannot be certified (appendix C, fig. 4). These studies therefore test support sufficiency at full coverage and cannot identify whether q ranks dispatch risk.

Proposition 1 (Per-candidate selective-risk admission)**.** *Condition on train/development data and fix one candidate (P, H, V, q), group-level violation rule, and finite grid Λ before observing calibration. If calibration groups are i.i.d. from the future-group distribution, then with probability at least $1 - \delta$, every $\eta \in \Lambda$ satisfies $r_\eta \le U_\eta$, where r_η is $\Pr(W^\eta = 1 \mid D^\eta = 1)$ when population admission is positive and is defined as zero otherwise. Consequently the selected $\hat{\eta}$ satisfies $r_{\hat{\eta}} \le \alpha$ whenever $U_{\hat{\eta}} \le \alpha$.*

Appendix A gives the proof. The guarantee is per fixed candidate and conditional on i.i.d. groups, which the cohorts assume rather than establish (section 7). Conditional on train and development data the number m of candidates reaching calibration is fixed, so a run is compiler-wide when $m = 1$ (Corollary 2) or when γ is divided by m; appendix C reports m and both bounds for every artifact.

4 EXPERIMENTAL SETUP

4.1 BENCHMARK AT A GLANCE

Every GitHub sample begins with one issue or pull-request number from a pinned public snapshot; the agent must make read-only calls and return exact, source-grounded fields. The three primary families use 132 discovery and 30 balanced held-out records each, with live provider calls, and each pairs a read-only evidence path with an exact answer contract — issue type, pull-request outcome,

Table 1: Primary live-provider results ($n = 30$ held-out records per family). "Interfaces" counts provider-visible tool interfaces. All conditions see identical records, model, and grader; *Manual* is hand-written with full knowledge of the workflow, and table 9 in appendix D prices it on these same resource axes.

	Exact held-out contract			Reduction of compiled vs. baseline (%)				
Family	Base	Compiled	Manual	Requests	Interfaces	Tokens	Latency	Cost
Issue-type routing	30/30	**30/30**	30/30	50.0	0.0	39.5	51.7	32.0
PR-outcome audit	30/30	**30/30**	30/30	75.0	66.7	80.8	73.0	75.3
Backlog-attention routing	29/30	**30/30**	30/30	74.8	66.3	81.4	68.9	75.1
Weighted total ($n = 90$)	89/90	**90/90**	90/90	66.6	44.2	63.1	64.2	58.7

or backlog attention route, each returning a category, a title, and a verbatim excerpt (table 11, appendix E).

Cross-repository time-forward extension. A narrower two-read PR-outcome task spans five frozen public repositories. Held-out records are strictly newer than discovery records, and selection is sealed before provider calls; repositories without an admitted artifact remain in the result. A balanced rerun on three repositories controls the `open/merged/closed_unmerged` mix.

External compiler substrates. NESTFUL, API-Bank, and executed BFCL v4 gold plans supply executable traces with the argument-level producer structure the compiler needs (Basu et al., 2025; Li et al., 2023; Patil et al., 2025). AppWorld adds a fourth and different one (Trivedi et al., 2024): its public gold solutions, executed on its pinned resettable backend, give 136 byte-exactly reproducible traces over 68 typed APIs and enough groups for eq. (7) to be satisfiable, and its released baseline runs supply held-out trajectories for the dispatch measurement. No model runs on any substrate, so none licenses an accuracy or efficiency claim and none is pooled with the GitHub families (appendix F.3).

Provider configuration. The primary studies use one provider and model — OpenAI `gpt-5.6-luna` via the `openai-agents` Responses interface at reasoning effort `low` — priced at list rates; the replications in appendix G use `gpt-6-luna` and Anthropic `claude-sonnet-5` under the same settings (appendix C). Conditions within a family differ in structure alone.

Conditions, metrics, and statistics. Each GitHub family compares the unchanged baseline B, the compiled artifact, and a hand-written pre-model program on the same records, so comparisons are paired rather than across cohorts. Quality is exact task-contract satisfaction; resources are requests, interfaces, tokens, latency, and estimated cost. We use paired exact McNemar tests, bootstrap intervals, and Wilcoxon signed-rank tests, and the only guaranteed multiplicity control is the Bonferroni term in eq. (7). All admission runs use $\alpha = 0.05$, $\delta = 0.1$, and $|\Lambda| = 11$ (appendix C).

5 Results

5.1 Primary workflows: preserved contracts, less work

Across the 90 held-out records, GAC satisfies 90/90 exact contracts and the unchanged agent 89/90 (table 1), reducing provider requests by 66.6%, tokens by 63.1%, latency by 64.2%, and cost by 58.7% (per-record values in table 6, appendix D). Issue-type routing admits only a two-read prefix; the two deeper workflows approach 75% fewer requests.

The discordant pair is one baseline-only miss (backlog record 5189, an ungrounded excerpt) against zero compiled-only misses (McNemar $p = 1$; one-sided 95% upper bound on compiled-only discordance 3.3%, or 2.2% over 210 records with the extension in appendix D); the same 30 records rerun on 2026-08-18 scored 30/30 in every arm (appendix G), so the miss is a stochastic baseline event and the result supports preservation on this cohort, not superiority or equivalence. Each artifact was admitted with 92 zero-violation calibration groups ($U_{\hat{\eta}} = 0.0498 \leq 0.05$), a bound conditional on i.i.d. groups. The certificate is compiler-wide for issue-type routing, where one candidate reached

Table 2: Where refusal happens, and what happens where it does not. Stage numbers are Alg. 1's; "slots" is the share of dependency slots with a recovered producer, and $n_{\max}$ is the largest family support against the 92 eq. (7) requires. "p/a/w" is pass/abstain/wrong on each substrate's *family-level* replay split, which is held out from a different fit than the admission split quoted in the text (appendix F.3).

Substrate	Traces	Slots (1)	Windows (2)	At floor (2)	Synth. (4)	Replay p/a/**w** (5)	$n_{\max}$ (6)	Adm.
NESTFUL	1,415	96.3%	1,207	32	12	24 / 12 / **0**	26	**0**
API-Bank	212	63.5%	37	8	2	0 / 2 / **0**	8	**0**
BFCL v4	200	68.0%	86	9	4	3 / 3 / **0**	15	**0**
AppWorld	136	63.2%	136	33	1	34 / 0 / **0**	**136**	**1**

calibration, and per-candidate for PR-outcome and backlog routing, where two did; correcting for both gives $U = 0.0569$, above α, so those two are certified compiler-wide only at $\alpha \geq 0.057$ (Corollary 2, table 4 in appendix C). On `gpt-6-luna` and on Anthropic `claude-sonnet-5`, same cohorts, re-discovery from each model's own traces re-derives two of the three artifacts and refuses the third; the transfer of the retained artifacts to `gpt-6-luna` keeps the structure with one compiled-only miss in 90 (appendix G).

Discovery reaches the manual ceiling: programs written with full knowledge of the workflow also reach 90/90 and, priced on the same axes (table 9), tie on requests, issue fewer interfaces, are marginally cheaper in tokens and dollars, and lose only on latency. We claim discovery and admission, not runtime dominance over hand-written code.

5.2 Time-forward transfer: compile or retire

On the narrower two-read pull-request outcome task across frozen public repositories (section 4), GAC completes 120/120 exact pairs on four of five repositories and retires `pytorch/pytorch` at admission, where no calibration group is accepted at any threshold, rather than loosening its gate (appendix F). The compiled program ran on 80/120 held-out records: every open-class record abstained on the induced verifier's `pr.state` hull, learned from discovery pools holding 0–3 open records, and ran the unchanged agent, so requests fall 44.4% overall (66.7% on dispatched records) and tokens 52.4%. A balanced three-repository rerun, with all three classes in discovery, dispatches on 180/180 and reaches 180/180 exact pairs at 66.7% fewer requests and 78.4% fewer tokens (table 13 in appendix F). A fixed two-read template is more efficient in the core protocol and tied in the rerun.

5.3 What binds: sample size, then deployability

The external substrates isolate the binding constraint (table 2). All four clear provenance, synthesis, and replay with *no* wrong execution, then NESTFUL, API-Bank, and executed BFCL gold plans retire, each settled arithmetically before the compiler ran. AppWorld's gold solutions reach $n_{\max} = 136$, making it the first external corpus where eq. (7) is satisfiable, and it admits one artifact at 92/92 zero-violation groups ($U_{\hat{\eta}} = 0.0498$) with 11/11 exact replay on its sealed admission split, under both pre-registered effect arms. Two features bound it: the gate is step-like and the admitted program is argument-free (appendix F.3).

Admissibility is not dispatchability. Gold solutions are not agent trajectories, so we also test the admitted region against AppWorld's 28 released baseline runs. Over 8,190 trajectories it is structurally eligible at the entry boundary in 37.9%, and the architecture explains the spread (table 14 in appendix F.3): full-code agents are eligible almost always, ReAct and plan-and-execute almost never. They open by reading API documentation, yet admitting that read as a prologue recovers only 2 of 4,680, because they rarely execute the region at all (appendix G); the measurement bounds φ from above.

6 Related Work

Guarded trace and agent workflow compilation. GAC adapts the tracing-JIT pattern—compile a hot path behind a guard, deoptimize on a miss (Bala et al., 2000; Hölzle et al., 1992)—to agents,

adding grounded arguments and declared effects (Lucassen & Gifford, 1988). Agent systems mine workflows, cache plans, compile orchestration, validate plans, or prune topology (Abuzakuk et al., 2026; Wang et al., 2025; Zhang et al., 2025; Wei et al., 2026; Winston et al., 2026; Li et al., 2026; Chen et al., 2026); DSPy optimizes prompts (Khattab et al., 2024), GEPA evolves them (Agrawal et al., 2026), RouteLLM changes models (Ong et al., 2025), LLMCompiler parallelizes calls (Kim et al., 2024). Unlike these, GAC treats recurrence only as a proposal: provenance, effects, position, contracts, and finite-sample risk each retain a veto (table 20 in appendix G).

Synthesis, risk control, and evaluation. Bounded search draws on programming by example, sketching, and provenance (Gulwani, 2011; Solar-Lezama et al., 2006; Cheney et al., 2009); admission combines Clopper–Pearson with fixed-grid learn-then-test (Clopper & Pearson, 1934; Angelopoulos et al., 2022). BFCL, ToolLLM, and AppWorld score tool use or interactive agents broadly (Patil et al., 2025; Qin et al., 2023; Trivedi et al., 2024) (appendix F.3).

7 Discussion and Limitations

The supported claim is narrow: repeated read-only prefixes can become guarded programs that preserve held-out contracts on the tested GitHub families and retire when evidence is inadequate. Across 750 compiled held-out episodes on the calibrated model in five live studies, one produced a compiled-only contract failure, an excerpt-fidelity miss on identical evidence (appendix D). Each limitation in appendix G is paired with a committed protocol, and appendix I maps every claim to its retained evidence.

Three constraints bound the claim. First, admission control is compiler-wide only when one candidate reaches calibration or the confidence budget is divided across all that do. That holds for issue-type routing and for every cross-repository family; PR-outcome and backlog routing calibrated two candidates on 92 groups, where the corrected bound is 0.057 (appendix C). Second, with $|\mathcal{K}| = 92$ the registered exact bound certifies only full coverage, so no experiment here demonstrates graded selectivity of q: the prospective protocol (appendix H) found the learned gate and a support-only comparator indistinguishable across five repositories and 240 held-out triples, and q is constant within a repository there because the entry state is one integer. Third, the binomial result is conditional on i.i.d. groups: the 92-record live calibration cohorts span 86–90 distinct creation days and 60–82 distinct authors per family, so a bound that treats a day or an author as one unit retires in every family (table 17 in appendix G), and AppWorld's task groups share a scenario's supervisor and database (appendix A prices the exposure).

External validity, dispatchability, and comparison. The richer evidence comes from one pinned repository snapshot and one calibrated model family; a second model family and a second provider each re-derive two of the three artifacts from their own traces and refuse the third, and every compiled-only miss observed is an excerpt-fidelity error of the model (appendix G). We show no superiority under drift, do not price manual authoring, and run no head-to-head against workflow-reuse systems, which make no admissibility decision to compare against; table 20 in appendix G prices the guards internally, and a pre-registered recurrence-only replay on issue-type routing passed 30/30 with one request per record, so on that cohort the refusal bought no measured quality (appendix G). Exact contracts also do not certify a downstream continuation, and writes, handoffs, hosted tools, undeclared effects, and removed-turn guardrails lie outside Proposition 1.

8 Conclusion

GAC treats recurrence as candidate evidence, not permission to rewrite an agent: it preserves exact held-out contracts while saving resources, re-derives the same artifacts on a second model and a second provider, retires all families on three external substrates, and admits a small, architecture-limited region on the fourth. Judge workflow optimizers by their evidence, refusals, and fallback, not only by the turns they remove.

AI Use Statement

The authors used generative AI tools to restructure and copy-edit the manuscript, suggest section organization, draft and revise paper text, critique methodological exposition and internal consistency, convert repository-grounded results into LaTeX tables, and assist in designing and implementing the experiment and analysis scripts used to produce the reported results. AI tools were not used to fabricate, hallucinate, or substitute for results: every study was executed against real systems (live GitHub repositories, recorded agent traces, and public benchmark datasets), and every reported number, citation, equation, and comparison was independently checked against the retained execution artifacts in the repository. The authors take full responsibility for the final content, the experimental design, and the validity of all reported results.

Ethics Statement

The main risk in removing model-mediated decisions is over-trusting a compiled prefix, and thereby accelerating unsafe automation or deleting a guardrail evaluation that happened to run at the removed boundary. The method mitigates this by compiling only read-only prefixes, treating unknown effects, approvals, handoffs, and writes as hard barriers, and defaulting to retirement or fallback when evidence is insufficient. All study data are public records (GitHub repository metadata) or public benchmarks; no personal or private data were collected. The artifact is not a license to bypass human review, legal or compliance controls, or safety guardrails; unsupported regimes are part of the result, not exceptions to hide.

Reproducibility Statement

Algorithms 1 and 2 give the compiler and admission protocols, Algs. 3 and 4 give provenance construction and runtime dispatch, appendix A proves Proposition 1, and appendix C records the registered statistical and synthesis settings. The accompanying artifact contains source manifests, raw result files, deterministic table and figure generation, and scripts for the live GitHub studies, the cross-repository extension, and all four compiler substrates: NESTFUL, API-Bank, executed BFCL, and executed AppWorld. Appendix F.3 states how the latter two obtain observed results, and appendix C gives their exact provider-free entry points. Before upload, these materials will be supplied as an anonymous archive with hashes and manifests preserved. The artifact pins the provider model string, the `openai-agents` and `openai` SDK versions, the effective generation settings, cache policy, the 120 s timeout and no-retry policy, the per-record condition order, concurrency, the token fields used for cost, the rate-card date, the tool and effect-catalog digests, and every split and candidate manifest (table 3); the second-model and second-provider replications pin their model strings, the Anthropic adapter's request mapping, and their protocols the same way.

References

Sami Abuzakuk, Anne-Marie Kermarrec, Rishi Sharma, Rasmus Moorits Veski, and Martijn de Vos. Optimizing agentic workflows using meta-tools. *arXiv preprint arXiv:2601.22037*, 2026. URL `https://arxiv.org/abs/2601.22037`.

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026. URL `https://arxiv.org/abs/2507.19457`. Oral presentation.

Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=TNcOrox3tQ`.

Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. Dynamo: A transparent dynamic optimization system. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 1–12, 2000. doi: 10.1145/349299.349303.

Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate, Mayank Agarwal, Maxwell Crouse, Yara Rizk, Kelsey Bradford, Asim Munawar, Sadhana Kumaravel, Saurabh Goyal, Xin Wang, Luis A. Lastras, and Pavan Kapanipathi. NESTFUL: A benchmark for evaluating LLMs on nested sequences of API calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 33538–33547, 2025. doi: 10.18653/v1/2025.emnlp-main.1702. URL https://aclanthology.org/2025.emnlp-main.1702/.

Yulang Chen, Haoxuan Peng, Jinyan Liu, Zichen Wen, Dongrui Liu, and Linfeng Zhang. AgentSlimming: Towards efficient and cost-aware multi-agent systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pp. 30064–30086, 2026. doi: 10.18653/v1/2026.acl-long.1387. URL https://aclanthology.org/2026.acl-long.1387/.

James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009. doi: 10.1561/1900000006.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404.

Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pp. 317–330, 2011. doi: 10.1145/1926385.1926423.

Headroom Labs. Headroom: Compressing tool outputs, logs, and retrieved context before they reach the LLM. Software repository, https://github.com/headroomlabs-ai/headroom, 2026. Apache-2.0; self-reported 60–95% token reduction on JSON payloads and 73% on a GitHub triage workload. Accessed 7 August 2026.

Urs Hölzle, Craig Chambers, and David Ungar. Debugging optimized code with dynamic deoptimization. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 32–43, 1992. doi: 10.1145/143095.143114.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=sY5N0zY5Od.

Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 24370–24391, 2024. URL https://proceedings.mlr.press/v235/kim24y.html.

Junyan Li, Zhang-Wei Hong, Maohao Shen, Yang Zhang, and Chuang Gan. FlowCompile: An optimizing compiler for structured LLM workflows. *arXiv preprint arXiv:2605.13647*, 2026. URL https://arxiv.org/abs/2605.13647.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187. URL https://aclanthology.org/2023.emnlp-main.187/.

John M. Lucassen and David K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 47–57, 1988. doi: 10.1145/73560.73564.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023. URL https://arxiv.org/abs/2307.16789.

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 404–415, 2006. doi: 10.1145/1168857.1168907.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076, 2024. doi: 10.18653/v1/2024.acl-long.850. URL https://aclanthology.org/2024.acl-long.850/.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 63897–63911, 2025. URL https://proceedings.mlr.press/v267/wang25bx.html.

Lei Wei, Qi Liu, Ruiyang Huang, Xiao Peng, Sicong Xie, Lanbo Lin, Chenhao Jiang, Yuanwu Xu, Tianyuan Yang, Jiayao Liu, Li Cai, Zhaolu Kang, and Bin Wang. EvoC2F: Compiling tool orchestration for efficient and evolvable LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://openreview.net/forum?id=ZSGB91kMOG.

Caleb Winston, Ron Yifeng Wang, Azalia Mirhoseini, and Christos Kozyrakis. Agent JIT compilation for latency-optimizing web agent planning and scheduling. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://arxiv.org/abs/2605.21470.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Qizheng Zhang, Michael Wornow, and Kunle Olukotun. Cost-efficient serving of LLM agents via test-time plan caching. *arXiv preprint arXiv:2506.14852*, 2025. URL https://arxiv.org/abs/2506.14852.

A Proof of Proposition 1

Condition on the train/development data, fix $\eta \in \Lambda$, and write $\gamma = \delta/|\Lambda|$. Let D_i^η and W_i^η be the group admission and violation indicators defined in section 2. Because the entire candidate and grid were frozen before calibration and groups are i.i.d., the pairs (D_i^η, W_i^η) are i.i.d. Conditional on $n_\eta > 0$ admitted groups, their violation count therefore satisfies $k_\eta \mid n_\eta \sim \text{Binomial}(n_\eta, r_\eta)$, where $r_\eta = \Pr(W^\eta = 1 \mid D^\eta = 1)$. Inverting the one-sided exact binomial test (Clopper & Pearson, 1934) gives, for $0 \le k_\eta < n_\eta$, the upper bound $U_\eta = \text{Beta}^{-1}(1-\gamma;\, k_\eta+1,\, n_\eta-k_\eta)$ of eq. (7), with $U_\eta = 1$ when $k_\eta = n_\eta$; either way $\Pr\{r_\eta > U_\eta \mid n_\eta\} \le \gamma$. When $n_\eta = 0$ the event is impossible under the convention that empty-admission thresholds receive the vacuous bound $U_\eta = 1$. Averaging over the random value of n_η therefore yields $\Pr\{r_\eta > U_\eta\} \le \gamma$ for each fixed threshold. Applying the union bound over the finite grid gives $\Pr\{\exists \eta \in \Lambda : r_\eta > U_\eta\} \le |\Lambda|\gamma = \delta$, i.e. simultaneous coverage with probability at least $1 - \delta$. On that event, every threshold whose upper bound is at most α satisfies the group-level target in eq. (4), including the threshold returned by Alg. 2. ■

Two scope conditions are worth restating. First, the statement is *per fixed candidate*: Alg. 1 may calibrate several families and retain survivors, and that outer search is not multiplicity-corrected — except when the search is frozen, per the corollary below. Second, i.i.d. source groups is an assumption about the calibration cohort, not a property the compiler verifies; the grouped split in Alg. 1 makes it plausible by construction but does not establish it under temporal drift.

Corollary 2 (Compiler-wide guarantee under frozen selection)**.** *If Alg. 1 runs with candidate freezing enabled — ranking on train data, synthesizing and challenging on train and development windows only, and halting the search the instant any candidate reaches calibration, whatever its outcome — Proposition 1's guarantee holds for the run's actual reported output with no adjustment to $\gamma = \delta/|\Lambda|$: no candidate multiplicity remains to correct for.*

Proof. Ranking, synthesis, the challenge test, and the frozen score model q are all functions of train and development windows alone, fixed before a single calibration window is inspected. The search reaches calibration for the top-ranked surviving candidate exactly once and halts immediately after that candidate's outcome, so no second candidate's program, guard, verifier, or score model is ever compared against calibration data. Proposition 1's precondition — one candidate fixed before observing calibration — is met by the run's entire search, not by a hypothetical single-candidate sub-procedure, so its conclusion transfers unchanged. ■ □

This is a corollary, not a new inequality: it is a restatement of Proposition 1 for a search that never lets a second candidate see calibration data. Conditional on train and development data, the number m of candidates that reach calibration is fixed before any calibration label is read. The cross-repository extension runs with freezing enabled and every sealed repository report shows one candidate reaching calibration, so Corollary 2 applies to those admissions directly. Issue-type routing did not freeze, but its second candidate retired at synthesis (an ungroundable slot) and never saw calibration, so $m = 1$ and the same argument applies. PR-outcome and backlog-attention routing each calibrated $m = 2$ candidates on the same 92 groups and kept the higher-removal survivor: their certificates are per-candidate at $\alpha = 0.05$, and under $\gamma = \delta/(m|\Lambda|)$ the retained zero-violation tables give $U = 0.0569 > \alpha$, a compiler-wide certificate only at $\alpha \ge 0.057$ or after 106 groups (table 4). The pre-registered repair, a fresh class-balanced 106-group calibration per family (`paper/supplementary/multiplicity-repair-protocol.md`), failed its own go/no-go check: after excluding every record used in a retained cohort, the pinned snapshot holds 33 unused `open` pull requests and 8 unused `owned` backlog issues against the 36 per class the design requires, so it was not run and the per-candidate wording stays.

Multiplicity accounting. For one fixed candidate and one fixed threshold η, $\Pr(r_\eta > U_\eta) \le \gamma = \delta/|\Lambda|$. For one fixed candidate over the whole grid, the union bound gives $\Pr(\exists \eta \in \Lambda : r_\eta > U_\eta) \le |\Lambda|\gamma = \delta$. For a set of m candidates fixed before calibration and each evaluated over Λ, choosing $\gamma = \delta/(m|\Lambda|)$ gives $\Pr(\exists\, a, \eta : r_{a,\eta} > U_{a,\eta}) \le \delta$; at $\alpha = 0.05$, $\delta = 0.1$, $|\Lambda| = 11$ the zero-violation requirement is 92 groups for $m = 1$ and 106 for $m = 2$ ($U = 0.0498$ versus 0.0569 at $n = 92$). `paper/scripts/frozen_candidate_coverage_simulation.py` checks the *single-threshold* quantity exactly, without Monte Carlo: for one candidate at one threshold and $n = 92$, the population rate that maximizes miscoverage is $r^* \approx 0.637$, where miscoverage is $\approx 0.00909 \approx \gamma$. Pooling m such

Algorithm 3 BUILDPATG — typed argument provenance. PATG annotates every argument slot and blocks nothing itself; MINEREGIONS (Alg. 1) rejects any window containing an UNGROUNDED or AMBIGUOUS slot, so a slot with no witness (a genuine model decision) and a slot with too many witnesses both retire the region rather than defaulting to the cheapest explanation.

Input: episodes $\mathcal{T}$; entry schema Θ; effect catalog C; transform library $\mathcal{L}$; ambiguity cap κ
Output: provenance graph G (witness and order edges) and slot-status map S

$G \leftarrow \emptyset$; $S \leftarrow \emptyset$
for all episodes $E = (z, M, e_{1:T}, y) \in \mathcal{T}$ **do**
  $\Sigma \leftarrow \{(\text{“}z\text{”}, \pi, v) : (\pi, v) \in \textsc{Flatten}(z),\ \pi \in \Theta\}$ ▷ only allowlisted entry paths are eligible
  for all call–result pairs (c_j, o_j) in $e_{1:T}$, in temporal order **do**
    for all slot names $s \in \text{keys}(\text{args}(c_j))$ **do**
      $v_{j,s} \leftarrow \text{args}(c_j)[s]$ ▷ the slot's value, distinct from its name
      if (c_j, s) is declared literal-only in Θ **then**
        $S[j, s] \leftarrow$ LITERAL; **continue** ▷ preserve the declared constant; infer no provenance
      end if
      $\mathcal{H} \leftarrow \textsc{BestPerSource}\{(\sigma, g) \in \Sigma \times \mathcal{L} : g(\text{value}(\sigma)) = v_{j,s},\ \text{depth}(g) \le 2\}$
      if $\mathcal{H} = \emptyset$ **then**
        $S[j, s] \leftarrow$ UNGROUNDED ▷ no witness: a model decision
      else if $|\mathcal{H}| > \kappa$ **then**
        $S[j, s] \leftarrow$ AMBIGUOUS
      else
        $S[j, s] \leftarrow$ GROUNDED; $G \leftarrow G \cup \{\sigma \xrightarrow{g} (j, s) : (\sigma, g) \in \mathcal{H}\}$ ▷ $1..\kappa$ single-source witnesses; synthesis commits to one
      end if
    end for
    $\Sigma \leftarrow \Sigma \cup \{(\text{var}(c_j), \pi, v) : (\pi, v) \in \textsc{Flatten}(o_j),\ \textsc{Eligible}(\pi, v)\}$ ▷ o_j witnesses only *later* slots
  end for
  for all event pairs (e_s, e_t) with $s < t$ sharing a resource where at least one writes it **do**
    $G \leftarrow G \cup \{e_s \prec e_t\}$ ▷ order edge in observed temporal order
  end for
end for
return (G, S)

candidates at the uncorrected per-candidate budget raises that single-threshold union to 1.8% at $m = 2$ and 13.6% at $m = 16$, while the corrected budget keeps it below 0.81% for every m tested. These are illustrative single-threshold figures; the grid-wide and candidate-wide certificates are the δ statements above, not these numbers. A companion seeded Monte Carlo (seed `20260822`, 200,000 replicates per cell) draws the 92 groups in correlated blocks instead of i.i.d.: realized miscoverage matches the exact value at zero correlation and reaches 56% against the 0.9% nominal single-threshold budget at the most clustered configuration tested (block size 46, one quarter of blocks shifted by 0.3), which is why the live studies' `min_days=min_principals=1` configuration is this guarantee's weakest link.

B Additional Algorithms

Algorithm 3 constructs the typed provenance graph that decides eq. (1). PATG annotates; MINEREGIONS blocks. The two blocking marks are asymmetric on purpose: a slot with no witness is a genuine model decision and a slot with more than κ witnesses is ambiguous, so the analysis never resolves an argument by picking the cheapest available explanation. PATG keeps up to κ single-source witnesses per grounded slot and SYNTHESIZE commits to the one source stable across every supporting group.

Algorithm 4 is the runtime counterpart of eq. (3). Cheap deterministic checks reject first and the calibrated gate runs only on what survives them, so the common case costs a registry lookup and a guard evaluation. Once an artifact resolves, the position check runs before any guard: any tool already observed in the episode, whether or not the compiled program uses it, returns the unchanged agent (eq. (2)). Exactly one exit path compacts; every clean rejection returns the unmodified baseline agent, and any failure whose reversibility cannot be attested raises an incident rather than feigning a rollback. "Baseline" is byte-exact only where a staging owner holds the commit boundary: the outer runner can restore byte-identical model input, whereas a model-boundary adapter cannot un-emit a response the host has already committed to conversation history.

Algorithm 4 DISPATCH — boundary-time admission. Cheap deterministic checks reject first and the calibrated gate runs only on what survives them. Exactly one exit path compacts; clean rejections return the unmodified agent, while an unattested failure or a refused commit raises an incident.

Input: entry state z; runtime context $X = (M', \mathcal{O}, \text{SNAP}, \text{STAGE})$ — runtime manifest, tools already observed in this episode, optional snapshot provider, staging owner; registry $\mathcal{R}$; effect catalog C; mode $m \in \{\text{off}, \text{shadow}, \text{live}\}$
Output: observations for the region, FALLBACK to baseline agent B, or INCIDENT

if $m = \text{off}$ **then**
  return FALLBACK ▷ conformance: model-visible input must be byte-identical
end if
$A \leftarrow \mathcal{R}.\text{RESOLVE}(X.M'.\text{compatibility}, X.M'.\text{partition})$
if $A = \bot \lor A.\text{lifecycle} \neq \text{ACTIVE} \lor \neg\text{VERIFYSIGNATURE}(A)$ **then**
  return FALLBACK
end if
if $X.\mathcal{O} \neq \emptyset$ **then**
  return FALLBACK ▷ eq. (2): any earlier tool call, by any tool, leaves position 0
end if
if $H_A(z, X.M') \neq 1$ **then**
  return FALLBACK ▷ manifest pins, isolation keys, typed hulls, effect set
end if
if $m = \text{live} \land \exists f \in \text{tools}(A.P) : C(f).\textit{quota_attested} \land X.\text{SNAP} = \bot$ **then**
  return FALLBACK ▷ a quota-bearing read cannot be discarded cleanly without a snapshot
end if
if $q_A(z) > \eta_A$ **then**
  return FALLBACK ▷ calibrated abstention on an uncertain input
end if
if $m = \text{shadow}$ **then**
  log would-dispatch; **return** FALLBACK ▷ observe coverage without changing behavior
end if
$S \leftarrow X.\text{STAGE}.\text{BEGIN}(X.\text{SNAP})$; $r \leftarrow \text{INTERPRET}(A.P, z, \text{FACADE}(C))$ ▷ the facade re-checks effects at execution time
if r raised a pre-commit error **then**
  return FALLBACK if S.ABORTANDATTEST(), else INCIDENT
end if
if $V_A(r) \neq 1$ **then**
  return FALLBACK if S.ABORTANDATTEST(), else INCIDENT ▷ live-outs, effect multiset, and call count are checked before use
end if
return r.observations if S.COMMIT(r.effects) succeeds, else INCIDENT ▷ commit refuses any effect outside the read classes; a refused commit is never a fallback

C CONFIGURATION

Live-provider configuration. Every primary live study calls one provider and one model: OpenAI `gpt-5.6-luna` through the `openai-agents` SDK's Responses interface. Model settings are fixed across all conditions at reasoning effort `low` and verbosity `low`, with parallel tool calls disabled so provider-visible interface counts are comparable between the baseline and a compiled prefix, and server-side response storage disabled. Sampling parameters are not exposed for this model; the SDK sends none, and the effective request manifest is table 3. The replications in appendix G run the same harness on `gpt-6-luna` (same settings) and on Anthropic `claude-sonnet-5` through an adapter over the official SDK (adaptive thinking at effort `low`, parallel tool use disabled, structured answers through the same schemas). Reported dollars are an *estimate*, computed by applying the published standard short-context list rates below — retrieved 2026-08-02 for `gpt-5.6-luna` and 2026-09-24 for the other two — to the token counts retained per record; they are not an account invoice and carry no negotiated or cached-tier discount beyond the cached-input line.

Table 3: Effective live-provider request manifest. Every row names its source: a retained `run` block, or a code constant with its file and line. Cache and cost fields are the ones the price formula consumes.

Key	Value
Provider / model	OpenAI `gpt-5.6-luna` via the `openai-agents` Responses interface
SDK versions	`openai-agents 0.19.2`, `openai 2.52.0`, Python 3.14.4, macOS-26.5.2 arm64
Model settings	reasoning effort `low`, verbosity `low`, `parallel_tool_calls=False`, `store=False`
Sampling	temperature, top-p, and seed are not exposed for this model; the SDK sends none
Turn and time limits	`max_turns` 8 (families) / 6 (cross-repository); 120 s per episode; no retries; failures kept in `failures[]`
Concurrency	discovery 6 (PR, backlog) / 8 (issue-type, cross-repository); held-out arms one at a time per record (PR, backlog, cross-repository); blocked Latin design, 8-way (issue-type)
Condition order	per-record permutation cycle over all six arm orders, five records per order
Cache accounting	fields `input_tokens`, `cached_tokens`, `cache_write_tokens`, `output_tokens`; no shared server state; cached tokens 0 in PR/backlog/cross-repository, non-zero in issue-type
Pricing	USD per 1M tokens: input 0.20, cached 0.02, cache write 0.25, output 1.20 (list rates, 2026-08-02); cost = (input − cached − write)·0.20 + cached·0.02 + write·0.25 + output·1.20
Data pins	`helmo/github-issues@e344be7b` parquet sha256 `09453eef…` (tracked); four cross-repository parquets pinned by sha256 in `source_manifest.json`
Tool and effect pins	per-family `effect_catalog_version` digest, `tools_hash`, `prompt_hash`, `policy_hash`, `entry_contract_version`
Run timestamps	one per study: 2026-08-04, 2026-08-05, 2026-08-05 (families); 2026-08-07 (core); 2026-08-22 (gate frontier); per-record timestamps not retained

Rate (USD per million tokens)	`gpt-5.6-luna`	`gpt-6-luna`	`claude-sonnet-5`
Input	0.20	0.10	2.00
Cached input	0.02	0.01	0.20
Cache write	0.25	0.125	2.50
Output	1.20	0.50	10.00

Because every condition within a family shares this model, these settings, the same records, and the same grader, the conditions differ in execution structure alone.

Admission budgets. All reported admission runs use risk budget $\alpha = 0.05$, confidence budget $\delta = 0.1$, and a fixed grid of $|\Lambda| = 11$ thresholds, which fixes the zero-violation sample requirement at $n_\eta \geq \ln(\delta/|\Lambda|)/\ln(1-\alpha) = 91.6$, i.e. 92 admitted groups; at exactly $n_\eta = 92$ and $k_\eta = 0$ the bound is $U_\eta = 1 - (0.1/11)^{1/92} = 0.0498 \leq \alpha$. Tightening the confidence budget to $\delta = 0.05$ would raise the requirement to 106 groups, which no study in this paper reaches, so $\delta = 0.1$ is the operative setting throughout and is reported as such rather than rounded to match α. A Bonferroni repair over $m = 2$ candidate families uses $\gamma = \delta/(m|\Lambda|)$ and lands on the same 106-group requirement, which is why PR-outcome and backlog routing carry per-candidate certificates while issue-type routing and the frozen cross-repository families are compiler-wide (table 4). The grid is $\Lambda = \{0.02, 0.05, 0.08, 0.11, 0.14, 0.17, 0.20, 0.25, 0.30, 0.40, 0.50\}$ and the ambiguity cap is $\kappa = 3$. Synthesis uses the 23-operator closed library of table 5 at depth ≤ 2; region mining uses $w_{\max}$ tool calls per window and requires support across independent scenario groups. Every bound in this paper is conditional on i.i.d. calibration groups.

Runtime capability vocabulary. `quota_attested` is a boolean field of a tool's signed effect declaration, not a capability flag: it marks a read whose invocation advances a counter (a rate quota or an audit log) that the staging snapshot must attest over before an attempt can be discarded as clean, so in live mode such a tool is not dispatched without a snapshot provider (Alg. 4). Position 0 of eq. (2) is identified at runtime with "no tool observation has been committed in this episode": any earlier tool call, whether or not the compiled program uses that tool, returns the unchanged agent, the runtime form of the compile-time `prefix_only` constraint. In the shipped facade every effect outside {PURE, READ-LOCAL, READ-EXTERNAL} is refused at commit and raises an incident; no post-commit path is reachable in the reported runs, and Alg. 4's commit line is that contract.

Table 4: Certificate level of every primary artifact. m is the number of candidates that reached calibration, fixed conditional on train and development data; U is the zero-violation upper bound at $n = 92$ under the per-candidate budget $\gamma = \delta/|\Lambda|$ and under the candidate-wide budget $\gamma = \delta/(m|\Lambda|)$. Rows are recomputed from the retained per-threshold tables (`compiler.artifact.gate.notes`) of the admitted artifacts. A provider-free recompilation from the sealed discovery checkpoints reproduces both admitted artifacts (identical ids, split digests, and bounds) and recovers the dominated two-read candidates' tables, which are also 92/0 at every $\eta \geq 0.14$ with $U = 0.0498$ (`paper/results/iclr_revision/recompiled_candidates_*.json`).

Family	m	n_η/k_η	$U_{\delta/\|\Lambda\|}$ / $U_{\delta/(m\|\Lambda\|)}$	Certificate at $\alpha = 0.05$	Groups needed
Issue-type routing	1	92/0	0.0498 / 0.0498	compiler-wide	92
PR-outcome audit	2	92/0	0.0498 / 0.0569	per-candidate; compiler-wide at $\alpha \geq 0.057$	106
Backlog-attention routing	2	92/0	0.0498 / 0.0569	per-candidate; compiler-wide at $\alpha \geq 0.057$	106
Cross-repository (7 admitted)	1	92/0	0.0498 / 0.0498	compiler-wide (frozen search)	92

Table 5: The closed transform library $\mathcal{L}$ (version `T-v1`). These 23 operators are the *entire* expressive power of the binder: a slot with no chain of length ≤ 2 over a single source is `ungroundable` and retires its region, so this table fixes what eq. (1) can witness. Parameters shown in parentheses are drawn from the observed trace, not free-searched. Every chain is typed, and a runtime type mismatch prunes it during the value-directed search rather than at apply time.

Class	n	Operators
Identity / coercion	5	`id`, `str`, `int`, `float`, `bool`
String	6	`lower`, `upper`, `strip`, `split(sep)[i]`, `join(sep)`, `fmt(template)`
Numeric	5	`add(c)`, `mul(c)`, `round(k)`, `sum`, `len`
Collection	6	`project(path)`, `filter(path op const)`, `first`, `last`, `sort(path)`, `topk(path,n)`
Temporal	1	`date_fmt(pattern)`
Total	23	

The library is closed and versioned deliberately. Every operator added widens Alg. 3's coverage and its spurious-match surface together, so growth is a two-sided trade rather than a free improvement; artifacts pin the library version and a runtime version mismatch is a dispatch barrier. The search is value-directed rather than an enumeration of $|\mathcal{L}|^2$ expressions: it runs breadth-first from the concrete source value and keeps chains whose denotation equals the target, which prunes by runtime type for free and collapses observationally equivalent chains.

The nonconformity score q. q is an ℓ_2-regularized logistic model over entry-observable features only — missing-field indicators, empirical-hull margin, temporal drift, and provenance ambiguity — standardized on the development split's own moments so no calibration statistic enters the fit. Its positive class is an *unproductive* development group: one the artifact does not carry cleanly to a verified result, which on $\mathcal{D}$ means a guard rejection, a verifier abstention, or a manifest mismatch. Wrong executions are not available as training signal at this point in Alg. 1, because ReplayAndChallenge at line 12 retires any candidate that produces one on $\mathcal{D}$; every candidate reaching FitScore at line 13 therefore has zero contract violations on the development split by construction. This is a design consequence rather than an oversight — the cascade is ordered so that deterministic checks reject before the statistical stage runs — but it does mean q is fitted on abstention-shaped supervision and asked at calibration to rank a different event, which is one plausible reason it discriminates weakly (section 7).

What cohort size permits the gate to certify. The zero-violation floor $n_\eta \geq 92$ constrains coverage, not just admission. Certifying selective coverage $c = n_\eta/|\mathcal{K}|$ requires $|\mathcal{K}| \geq \lceil 92/c \rceil$: 92 groups for $c = 1$, 115 for $c = 0.8$, 132 for $c = 0.7$, 184 for $c = 0.5$. Every cohort in this paper fixes $|\mathcal{K}| = 92$,

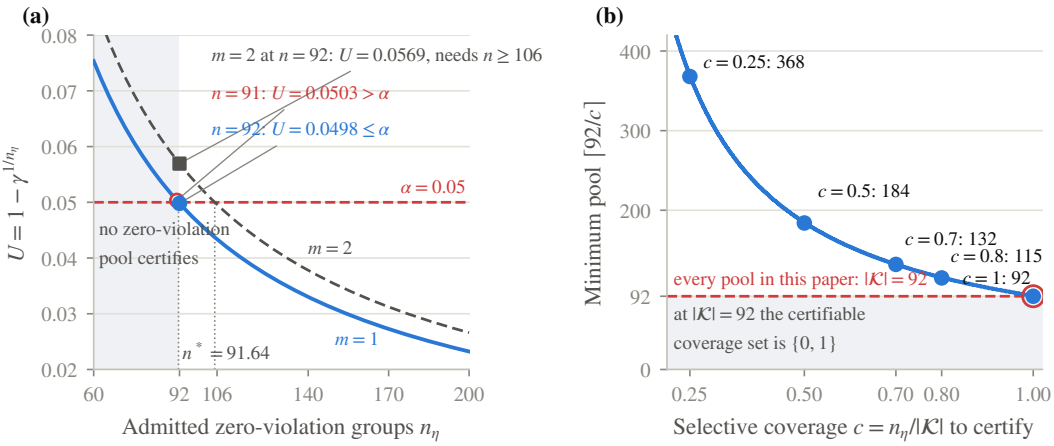


Figure 4: Why a 92-group pool certifies only full coverage. Left: the zero-violation bound $U(n) = 1 - (\delta/|\Lambda|)^{1/n}$ at $\delta = 0.1$, $|\Lambda| = 11$ crosses $\alpha = 0.05$ between $n = 91$ (0.0503) and $n = 92$ (0.0498). Right: the smallest pool $\lceil 92/c \rceil$ that can certify selective coverage c; every calibration pool in this paper is 92, so the certifiable coverages are 0 and 1 whatever q ranks.

which is the smallest pool at which eq. (7) is satisfiable at all. At that size the admissible coverage set is exactly $\{0, 1\}$: abstaining on one group leaves $n_\eta = 91$ and $U_\eta = 1 - (0.1/11)^{1/91} = 0.0503 > \alpha$. The step-like gate profiles reported in section 5.3 and appendix H are therefore consistent with two different worlds — a score that cannot rank, and a cohort too small to certify a ranking — and these experiments do not separate them. Two intermediate coverages were *observed* and neither is certifiable: `huggingface/datasets` accepts 10/92 groups at $\eta \leq 0.08$ in the gate-frontier study ($U = 0.375$), and `pandas-dev/pandas` accepts 90/92 at $0.08 \leq \eta \leq 0.20$ in the balanced rerun ($U = 0.0509$). A frontier study must first buy calibration groups (fig. 4). The implementation also supports a minimum-distinct-days requirement, which the single-snapshot live study does not exercise (`min_days = 1`). Two terms of eq. (5) are coarser in the implementation than the notation suggests: ρ_{eff} is approximated by the fraction of tool names containing a namespace separator rather than read from the effect catalog, and $|F|$ is substituted by the argument-slot count of the first observed window because ranking precedes synthesis. Both appear only in a ranking heuristic — no admission decision depends on either, and the effect catalog is consulted where it is load-bearing, in qualification and in the runtime facade.

The two executable-gold external substrates are provider-free after their pinned sources are prepared. Their exact compiler entry points are:

```
B=paper/scripts/bfcl_compiler_benchmark.py
python "$B" --source-root "$BENCHMARK_SOURCE_ROOT"
A=paper/scripts/appworld_compiler_benchmark.py
python "$A" --source-root "$BENCHMARK_SOURCE_ROOT" \
  --appworld-python "$APPWORLD_VENV/bin/python"
D=paper/scripts/appworld_dispatch_preflight.py
python "$D" --source-root "$BENCHMARK_SOURCE_ROOT"
```

The AppWorld environment uses Python 3.12 with package version `0.1.3.post1` and data version `0.1.0`; the signed source manifest and requirements file pin the remaining acquisition inputs. The BFCL source is pinned to revision `6ea57973c7a6`. These commands regenerate only aggregate, non-sensitive evidence; no model or provider call is made.

D Paired Statistics for the Primary Families

Latency validity. Table 7 gives the paired differences behind table 1. Latency is wall-clock provider evidence and is secondary to requests and tokens. In the PR-outcome and backlog fam-

Table 6: Absolute per-record resources behind the percentages of table 1: mean / median over the 30 held-out records of each family and over all 90. Estimated dollars are list-price estimates (appendix C); the entire 90-record compiled evaluation cost an estimated $0.028 against $0.068 for the baseline. Cells are mean / median.

Family	Arm	Requests	Interfaces	Tokens	Latency (ms)	Cost (USD×10^{-4})
Issue-type routing	Baseline	4.00	3.00	4,259.4 / 4,320	6,160 / 5,402	8.72 / 8.74
	Compiled	2.00	3.00	2,576.5 / 2,602	2,973 / 2,881	5.93 / 5.74
	Manual	2.00	1.00	1,780.8 / 1,734	3,251 / 2,856	5.45 / 5.49
PR-outcome audit	Baseline	4.00	3.00	2,718.2 / 2,676	5,329 / 4,839	6.79 / 6.66
	Compiled	1.00	1.00	521.5 / 462	1,440 / 1,297	1.68 / 1.51
	Manual	1.00	1.00	522.2 / 458	1,827 / 1,364	1.69 / 1.51
Backlog-attention routing	Baseline	3.97	2.97	2,844.9 / 2,816	6,428 / 5,163	7.29 / 7.10
	Compiled	1.00	1.00	529.9 / 468	2,000 / 1,557	1.82 / 1.62
	Manual	1.00	1.00	535.5 / 479	1,695 / 1,501	1.88 / 1.70
All (n = 90)	Baseline	3.99	2.99	3,274.2 / 2,907	5,972 / 5,163	7.60 / 7.42
	Compiled	1.33	1.67	1,209.3 / 615	2,137 / 1,957	3.14 / 2.05
	Manual	1.33	1.00	946.2 / 616	2,258 / 1,913	3.01 / 2.18

Table 7: Per-record paired differences (compiled minus baseline) for all three families, with 10,000-sample bootstrap intervals (seed 20260802) and two-sided Wilcoxon signed-rank tests over $n = 30$ paired held-out records. Latency is reported as the mean and as the median paired difference, each with its interval; cost intervals are in USD. Provider-request differences are deterministic by construction (every pair removes the same number of turns), so no rank test is reported for them: the statistic would measure the degeneracy rather than the effect. The backlog baseline's one three-request record (5189) is the only exception and appears in table 8.

Family	Endpoint	Paired difference (95% CI)	p
Issue-type	Total tokens	$-1,682.9$ $[-1,772.5, -1,596.7]$	1.9×10^{-9}
	Wall latency (mean)	$-3,187$ ms $[-3,758, -2,660]$	1.9×10^{-9}
	Wall latency (median)	$-2,716$ ms $[-3,532, -2,309]$	—
	Estimated cost	$-\$2.79 \times 10^{-4}$ $[-3.13, -2.41] \times 10^{-4}$	3.7×10^{-9}
PR-outcome	Total tokens	$-2,196.7$ $[-2,211.1, -2,182.5]$	1.7×10^{-6}
	Wall latency (mean)	$-3,889$ ms $[-4,499, -3,346]$	1.9×10^{-9}
	Wall latency (median)	$-3,476$ ms $[-3,736, -3,245]$	—
	Estimated cost	$-\$5.11 \times 10^{-4}$ $[-5.22, -5.02] \times 10^{-4}$	1.7×10^{-6}
Backlog-attention	Total tokens	$-2,315.0$ $[-2,366.0, -2,224.1]$	1.7×10^{-6}
	Wall latency (mean)	$-4,428$ ms $[-5,400, -3,558]$	1.9×10^{-9}
	Wall latency (median)	$-3,619$ ms $[-4,201, -3,091]$	—
	Estimated cost	$-\$5.47 \times 10^{-4}$ $[-5.66, -5.23] \times 10^{-4}$	1.7×10^{-6}

Table 8: Exact-contract discordance per family and pooled. "Baseline-only" counts records the unchanged agent passed and the compiled arm failed; "compiled-only" the reverse. The upper bound is the one-sided 95% Clopper–Pearson bound on the compiled-only failure rate.

Family	n	Both pass	Baseline-only	Compiled-only	McNemar p	Upper bound
Issue-type	30	30	0	0	1.00	9.5%
PR-outcome	30	30	0	0	1.00	9.5%
Backlog-attention	30	29	0	1	1.00	9.5%
Pooled	90	89	0	1	1.00	3.3%

ilies each held-out record ran its three arms one at a time on one host, cycling through all six arm orders (five records per order), with no warm-up, a 120 s per-episode timeout, and no retries; the issue-type family used a blocked six-permutation Latin design with eight-way concurrency within each block. No server-side state is shared between arms (`store=False`); cached-input tokens are zero on every PR-outcome and backlog record and are priced separately where they occur (`paper/results/cache_accounting.json`). Recomputing the compiled-versus-baseline wall-latency reduction within each of the six arm orders gives reductions between 43.2% and 78.6%

across the eighteen permutation–family cells (every cell positive), and provider-response latency reproduces the wall-latency reductions to within 0.3 percentage points, so the ordering design, not service variance, carries the effect.

Table 9: The hand-written comparator priced on the same resource axes as table 1, as percentage reduction against the same baseline and the same held-out records. *Manual* is written with full knowledge of the workflow, so this is the pessimistic comparison for the compiler: it ties on provider requests, issues fewer tool interfaces, and is marginally cheaper in tokens and dollars, while compilation leads only on wall latency (bold). The interface gap is concentrated in issue-type routing, where the compiled prefix covers two reads but the agent still issues the third. We therefore claim automated discovery and guarded admission, not runtime dominance over code written by someone who already knows the workflow.

Condition	Requests	Interfaces	Tokens	Latency	Cost
Issue-type routing					
Compiled	50.0	0.0	39.5	**51.7**	32.0
Manual	50.0	66.7	58.2	47.2	37.5
PR-outcome audit					
Compiled	75.0	66.7	80.8	**73.0**	75.3
Manual	75.0	66.7	80.8	65.7	75.2
Backlog-attention routing					
Compiled	74.8	66.3	81.4	68.9	75.1
Manual	74.8	66.3	81.2	**73.6**	74.2
Weighted total (n = 90)					
Compiled	66.6	44.2	63.1	**64.2**	58.7
Manual	66.6	66.5	71.1	62.2	60.4

An extended issue-type held-out cohort. A pre-registered extension (`paper/supplementary/issue-type-extended-heldout-protocol.md`) ran the retained issue-type artifact on 120 further unused records of the pinned snapshot on the calibrated model, balanced 40/40/40 over `bug`, `enhancement`, and `other` (the snapshot holds 16 exclusive `question` issues, 10 already used, so the retained three-way mix could not be reproduced), with the three retained arms in a balanced Latin order and the artifact recompiled provider-free under the current tracer pin (identical id, program, splits, and 92/0 gate). By the decision rule fixed in advance the compiled-only miss comes first: record 2737, where the compiled arm quoted an error line with backticks the source does not contain; the baseline missed record 3040 (a doubled space) and all three arms missed record 3968. Exact contracts were 118/120 baseline, 118/120 compiled, and 117/120 hand-written (table 10); the artifact dispatched on 119/120 records and abstained cleanly on record 5102 on the induced verifier's cardinality clause, running the unchanged agent. The one-sided 95% bound on compiled-only failures is 3.9% on these 120 records and 2.2% pooled with the 90 primary records (1/210); the pre-registered targets of zero misses, 120/120 dispatch, and a pooled bound at or below 1.5% were all missed and are reported as such. Five of the 120 records had appeared in the gate-frontier pilot's cohort because the exclusion scan omitted that directory; none of the missed or abstained records is among them, and without them the counts are 113/113/112 of 115 with bounds of 4.1% and 2.3%. Requests fell 50.0%, tokens 39.5%, latency 42.5%, and cost 32.8% against the same-model baseline, matching the primary family within a point.

E Additional Experimental Detail

Primary workflow families. The three-family result is the main paper's strongest live evidence because it combines distinct tool vocabularies, exact task graders, balanced held-out cohorts, and provider-backed execution. The issue-type family intentionally retains a shallower two-read prefix; the newer pull-request and backlog families admit deeper pre-model compaction. This difference explains the lower resource savings on the first family and is part of the evidence boundary, not noise to average away.

Table 10: Extended issue-type held-out cohort on the calibrated model: per-record means over 120 unused records with a disclosed class mix (40 `bug` / 40 `enhancement` / 40 `other`). The compiled-only miss (record 2737) and the pooled bound are the pre-registered primary endpoints.

Arm	Exact	Requests	Tokens	Latency (s)	Cost (¢)
Unchanged agent	118/120	4.00	4,381	6.70	0.090
Compiled two-read prefix (GAC)	118/120	2.00	2,649	3.85	0.061
Hand-written bundle	117/120	2.00	1,813	3.68	0.055
Compiled-only failures: 1/120 here (one-sided 95% bound 3.9%); 1/210 pooled with the primary cohorts (bound 2.2%)					

Table 11: Primary GitHub benchmark at a glance. Each sample supplies one record number from a pinned public snapshot; every listed answer field is checked against the source record.

Task	Read-only evidence path	Exact answer contract
Issue type	`issue_number` → record, official labels, comments	Title, state, label-derived category, verbatim excerpt
PR outcome	`pr_number` → PR record, merge status, discussion	`open`, `merged`, or `closed_unmerged`, plus title and comment excerpt
Backlog attention	`issue_number` → open-issue record, assignees, discussion	`owned`, `discussed_unowned`, or `awaiting_first_response`, plus title, owner, and excerpt

Cross-repository extension. The cross-repository study is narrower than the primary workflow families. Its task uses two read-only tools and a simpler exact outcome contract, which is precisely why a fixed template can remain tied or even more efficient than the learned artifact. We keep the result in the main text because it addresses a reviewer-critical scope question—whether guarded specialization survives beyond one repository—while being explicit about the reduced task complexity.

The position invariant, empirically. An early pilot dispatched a compiled *suffix* region at the entry boundary, violating eq. (2). Because the runtime resolves artifacts before the agent has made the calls the region assumed had already happened, the pilot reordered and duplicated tool calls. The failure is a semantic mismatch between compilation-time position and resolution-time position, not a tuning error, which is why eq. (2) is a hard constraint in the deployed compiler rather than a scoring penalty.

Omitted from the main claim. The repository retains additional studies that are useful scientifically but less suitable for the main narrative:

- a natural-order issue study that exposed a factual continuation miss for a more aggressive three-read artifact,
- exploratory guarded composite synthesis and a follow-up comparison to a fair pre-model manual program,
- a bounded GEPA prompt-optimization study in which GEPA retained its seed, so the combined arm is a replication rather than evidence of interaction, and
- a portfolio pilot and controlled demonstration suite for runtime boundary coverage, including workloads where removing turns *increases* cost through prefix-cache fragmentation.

These studies remain important to the broader artifact, but they either use a weaker risk level, address a narrower engineering question, or are explicitly exploratory.

F Evidence Breadth and Boundary Audits

The main text uses the nine-page budget for the registered live studies, the time-forward extension, and the four compiler-measured external substrates. This appendix makes the retained breadth visible without pooling incomparable cohorts or promoting screened and exploratory work to deployment evidence.

Table 12: Held-out dispatch in the cross-repository studies. "Dispatched" counts records on which the compiled program ran; the remainder abstained on the named verifier clause and ran the unchanged agent, which is why the request reduction is 44.4% under the core protocol and 66.7% under the balanced one for the same two-read artifact.

Protocol	Repository	Held-out	Dispatched	Fallback clause	Req. ↓
Core	`huggingface/datasets`	30	20/30	`range:pr.state` (10)	44.4%
Core	`pandas-dev/pandas`	30	20/30	`range:pr.state` (10)	44.4%
Core	`psf/requests`	30	20/30	`range:pr.state` (10)	44.4%
Core	`pytorch/pytorch`	—	RETIRE	—	—
Core	`streamlit/streamlit`	30	20/30	`range:pr.state` (10)	44.4%
Balanced	`pandas-dev/pandas`	60	60/60	—	66.7%
Balanced	`psf/requests`	60	60/60	—	66.7%
Balanced	`pytorch/pytorch`	60	60/60	—	66.7%

F.1 Repository-level time-forward outcomes

Why `pytorch/pytorch` retires in the core protocol and admits in the balanced rerun. The core retirement is attributable to one stage. Its compile report records 116 windows accepted, no windows blocked and none blocked by tool, one candidate mined, and a single rejection at `calibrate:retire` — stage 6 — with the message that the tightest attainable Clopper–Pearson upper bound over the grid is $1.000 > \alpha$. A bound of exactly 1 is eq. (7)'s degenerate branch, reached when no calibration group is admitted at a threshold or when every admitted group violates, so the candidate never reached a countable zero-violation cohort at any η. Provenance, synthesis, and contract induction are therefore not implicated: the region was groundable and the program was synthesized. What differs between the protocols is the class composition of the pool the artifact was fitted and calibrated on. The core protocol draws discovery from the older eligible pool with class balance explicitly not enforced, which for this repository yields 84 `closed_unmerged`, 30 `open`, and 2 `merged` records out of 116, while the sealed held-out split is an exact round-robin of 10/10/10. The deployment distribution the gate is asked to certify therefore contains a third `merged` records against a discovery pool containing under 2% of them. The balanced rerun enforces the `open`/`merged`/`closed_unmerged` mix at selection and the same task admits. We report both rather than only the rerun: the core outcome is the protocol behaving as designed under a skewed pool, and it is the clearest instance in the paper of the gate refusing on evidence composition rather than on trace structure. A provider-free recompilation from the retained discovery checkpoint (`paper/results/iclr_revision/pytorch_core_recompile.json`) reproduces the retirement and recovers the per-threshold rows the original run truncated: $n_\eta = 0$ at every $\eta \in \Lambda$, so no calibration group was accepted at any threshold and the bound is the degenerate branch of eq. (7). The rows identify the branch, not its cause; the class-composition explanation is consistent with the balanced rerun's admission but is not established by them.

Held-out dispatch and cohort provenance. Table 12 separates calibration coverage from held-out dispatch. In the core protocol the artifact dispatched on 20 of 30 records in every repository; the ten open-class records abstained with reason `range:pr.state`, because the verifier's categorical hull, induced on discovery pools with 2, 0, 3, and 0 open records, contained only `closed`, and the abstention is a clean fallback that runs the unchanged three-request agent. The balanced rerun's 40/40/40 discovery pools induce the hull {`closed`, `open`} and dispatch 60/60 per repository, which is the whole difference between 44.4% and 66.7%. The core summary was produced in two invocations: the first, with a five-repository completeness gate, stopped when `pytorch/pytorch` retired; the second accepted four repositories and 120 pairs and reused the four completed per-repository results unchanged. The three cross-repository cohorts are drawn from the same frozen snapshots without mutual exclusion: the balanced rerun shares 13, 25, and 8 of the 30-record core held-out sets, and the gate-frontier cohort of appendix H shares 130/150 held-out and 442/460 calibration records with the core cohort (`paper/results/iclr_revision/cohort_overlap_audit.json`).

The core result succeeds on four repositories (120/120 held-out pairs) and retains the fifth as a compile-time retirement after 116 exact discovery traces. The balanced rerun has a distinct three-repository cohort and reaches 180/180 held-out pairs. These rows show breadth across repositories

Table 13: Disaggregated cross-repository time-forward outcomes behind section 5.2. The core protocol uses four completed repositories and one compile-time retirement; the balanced rerun changes the selected repositories and class mix, so it is reported separately rather than pooled with the core protocol. Reductions compare the compiled condition with the paired unchanged baseline.

Protocol	Repository	Held-out	Exact	Req. ↓	Tokens ↓
Core	huggingface/datasets	30	30/30	44.4%	52.2%
Core	pandas-dev/pandas	30	30/30	44.4%	52.5%
Core	psf/requests	30	30/30	44.4%	52.3%
Core	streamlit/streamlit	30	30/30	44.4%	52.7%
Core	pytorch/pytorch	—	RETIRE	—	—
Balanced	pandas-dev/pandas	60	60/60	66.7%	78.6%
Balanced	psf/requests	60	60/60	66.7%	78.7%
Balanced	pytorch/pytorch	60	60/60	66.7%	78.0%

drawn from shared pools — the balanced rerun shares 13, 25, and 8 of the 30-record core held-out sets — not independent evidence for an average effect: all runs share the same provider family and the same narrow two-read outcome task.

F.2 Continuation audit

The earlier natural-order three-read study is excluded from the registered $\alpha = 0.05$ headline because it used $\alpha = 0.10$ and exposed a downstream continuation error. Its retained counterfactual replay nevertheless tests a boundary the exact tool contract cannot see: 17/18 candidate continuations pass unchanged, while issue 6602 has a comment-evidence mismatch. Checked rendering repairs that one case, yielding 18/18 final contracts. This audit makes no provider calls and does not estimate live agent quality; it establishes only that the continuation check caught a concrete failure the compacted tool trace alone would miss.

F.3 External evidence map

Table 15 records every named external benchmark in the retained evidence register and its admissible interpretation.

Obtaining observed results from BFCL. BFCL's reference plans carry no tool results, which is why the plan-format row and the compiler row are listed separately. Its multi-turn categories, however, ship an executable stateful backend: each task publishes an `initial_config` snapshot and the classes it involves, so executing the *official gold plan* through the upstream helper yields observed results, an entry snapshot, and an in-process replay oracle, with no model in the loop. Five preconditions fail the run closed rather than publishing: an undeclared method, any upstream execution error, a read-like declaration observed mutating state, a compilable declaration observed advancing the scenario RNG, and any inexact re-execution. All held: 200 tasks and 1,142 calls executed cleanly and a second independent pass reproduced every result byte for byte, executing turn-at-a-time where the first pass executed call-at-a-time.

Why the catalog is signed rather than inferred. The same run audits the declarations empirically by snapshotting every involved instance around every call. It observed 1,060 state-mutating calls over 44 of the 81 methods and 94 calls that advanced the seeded scenario RNG, with zero cases of a read-like declaration mutating state or a compilable declaration advancing the RNG. One case is worth naming: a method whose name and docstring present it as a cost getter mutated its instance's internal lookup on all 36 of its calls and is therefore declared a write. A name- or docstring-derived effect label would have admitted a write into a compiled region. Two further methods are read-like but carry no capabilities, because one derives an age from the host date and the other draws from — and advances — the seeded RNG; they are barriers by declaration and account for 35 suppressed spans. Write barriers suppress 2,802 spans in total, against 45 for entry-schema admissibility, 31 for slot ambiguity, and 5 for partial runs.

Table 14: Structural dispatch eligibility of the admitted AppWorld artifact on the 28 released official baseline runs. Eligibility is the admitted program's exact call sequence at normalized position 0; it is an upper bound on φ, because the manifest check and the calibrated gate are not evaluated and the released logs retain API calls rather than model boundaries. Trajectories repeat tasks across runs and are descriptive rather than independent samples. A miss abstains to the unchanged agent.

Released baseline architecture	Runs	Trajectories	Eligible at pos. 0	Rate
Full code + reflection	8	2,340	2,339	1.000
Iterative parallel function calling	4	1,170	762	0.651
Plan and execute	8	2,340	1	0.000
ReAct	8	2,340	0	0.000
Pooled	28	8,190	3,102	0.379

Obtaining observed results from AppWorld, and why it is the reachable case. AppWorld's shipped `api_calls.json` records method, url, and request data but no responses, so it fails closed for the same reason BFCL's plans do. Executing the official `compiled_solution_code` against the pinned `0.1.3.post1` backend supplies the results; each task's own `specs.json` supplies the entry snapshot; a second execution supplies the replay oracle. Calls are captured at the single dispatch point every `apis.<app>.<api>(...)` call passes through, so the recorded identity is a semantic app/API pair with typed keyword arguments and a decoded JSON result. Of the 147 public train+dev gold solutions, 136 execute cleanly; 11 dev solutions raise against the shipped data version and are reported as upstream compatibility outcomes rather than dropped. The second pass reproduced 7,070/7,070 results byte for byte, minted access tokens included, because AppWorld freezes each task's clock.

The empirical audit attributes every `INSERT`/`UPDATE`/`DELETE`/`CREATE`/`DROP` statement to the enclosing call. It observed 1,320 mutating calls, zero read-like declarations mutating, and—unlike BFCL—zero APIs that mutate on some calls but not others. Every `GET` was clean on every call and every `DELETE` and `PATCH` mutated on every call, so the effect boundary is crisp; the five `*.login` endpoints are the only non-`GET` APIs that never mutate, which is why they are the single contestable declaration and why the run is pre-registered in two arms. Groups are tasks, not scenarios: AppWorld's three variants of a scenario share a supervisor and a database, so the bound is conditional on task-level independence, which this corpus does not establish—the same conditional the live studies carry, neither weakened nor repaired here. The calibration share is fixed at exactly the 92 groups the bound needs, since a larger set would make the bound easier and a smaller one would make it unreachable. Two held-out replay figures appear for AppWorld and they count different partitions, which table 2 flags but does not have room to explain. The 34/0/0 in that table is the test side of the *family* partition used to challenge a mined candidate — 102 train and 34 test groups, the same accounting every substrate row reports. The 11/11 quoted in section 5.3 is the sealed test split of the 11/92/11 dev/calibration/test partition Alg. 2 consumes for admission. Both are genuinely held out; they are held out from different fits, and neither is a subset of the other's training side.

Admissible is not dispatchable. Table 14 separates the two. The admitted region is eligible on essentially every full-code trajectory and essentially no ReAct or plan-and-execute trajectory, because those architectures open by reading API documentation—3,752 of the 8,190 released trajectories begin with an API-description call—so the compiled prefix is no longer at position 0 and (2) refuses. A further 61 trajectories contain the program somewhere other than position 0, which the same invariant refuses. Full-code agents are eligible on 2,339/2,340 trajectories and iterative parallel function-calling agents on 762/1,170. Two secondary observations: the rate is flat across task difficulty (0.377 on `test_normal` against 0.380 on `test_challenge`), so this is a calling-discipline property rather than a workload property; and the maximal compilable depth is 2 on the eligible families, so the admitted two-step program sits at the ceiling the effect catalog permits rather than short of it. On the ineligible families the measured depth is a floor, not an estimate, because the API-documentation call is undeclared in the signed catalog and unknown effects are not reads.

Table 15: Disposition of every retained external benchmark family. Only the four compiler-measurement rows support compiler claims; the remaining rows document why broader benchmark names are not silently converted into GAC results. BFCL appears twice because its reference plans and its executed gold plans are different evidence.

Benchmark	Evidence tier	Retained result / claim boundary
NESTFUL	Compiler measurement	1,415 traces; 24 / 12 / 0 replay; all families retire.
API-Bank	Compiler measurement	212 traces; 0 / 2 / 0 replay; all candidates retire.
BFCL v4 (plans)	Gold-plan checker	200/200 valid plans; plan format only.
BFCL v4 (executed)	Compiler measurement	200 traces, 1,142/1,142 exact re-execution; 3 / 3 / 0 replay; all families retire.
AppWorld (executed)	Compiler measurement	136/147 traces, 7,070/7,070 exact re-execution; $n_{\max} = 136 \geq 92$; one artifact admitted at $U = 0.0498$; four candidates retire at the margin.
ToolSandbox	Live-provider subset	One simulated scenario; compiler not run.
τ^2/τ^3	Live-provider subset	0/4 pass; compiler not run.
ToolBench	Adapter smoke	Ten fixtures; full data/backend unavailable.
AgentBench	Adapter/preflight	Services and Freebase data gated.
GAIA	Access preflight	Authorization denied; no metric.
SWE-bench Verified	Task adapter	Official run host-gated.
BrowseComp	Live-web subset	1/3 correct, 28 searches; bypassed by the compiler.

Table 16: Gate-behavior inventory from the retained threshold analysis. A partial frontier would show graded coverage as the threshold changes; none appears among the seven current registered artifacts.

Evidence set	Artifacts	Partial frontiers	What it licenses
Current registered $\alpha = 0.05$	7	0	Exact per-candidate certificates; all are step-like.
Exploratory or looser-α	3	2	Diagnostic evidence only; not the registered claim.
NESTFUL refusal	12 synthesized	0	$n_{\max} = 26 < 92$; no admission.
API-Bank refusal	2 synthesized	0	$n_{\max} = 8 < 92$; no admission.
BFCL v4 refusal	4 synthesized	0	$n_{\max} = 15 < 92$; no admission, pre-registered.
AppWorld admission	1 admitted / 4 retired	0	$n_{\max} = 136 \geq 92$; the one reachable external bound. Admitted gate is step-like; retirements are marginal ($U = 0.051$ at 89 and 90 eligible groups).

F.4 The registered gate is a support threshold

This audit strengthens the evidentiary record precisely by preserving the negative result: the registered gate behaves as, and at $|\mathcal{K}| = 92$ can only behave as, a support threshold rather than a demonstrated risk–coverage frontier (table 16); the question of q's ranking quality is open, not answered negatively. The exploratory partial frontiers therefore remain diagnostic rather than evidence for the main claim.

G Extended Limitations

Repairs for non-exchangeable groups. Section 7 states that the cluster-aware repairs all cost effective sample size. Concretely, three are compatible with eq. (7) and none is exercised here: enforcing temporal separation at calibration (`min_days`$> k$) so a burst of related records cannot populate one split; replacing the per-group binomial count with a cluster-level or block-bootstrap bound that treats a correlated burst as a single unit; and carrying the cluster structure into the conformal step directly. Each shrinks the number of independent units below the 92 that the bound already demands, which is why none was affordable on these cohorts and why a larger corpus is the precondition for all

Table 17: Effective independent units in the 92-record calibration cohorts, joined to the pinned snapshot: distinct creation days, months, and authors, and the zero-violation bound the registered budget would give if each cluster were one unit (admits only when the count is at least 92).

Cohort	n	Days	Months	Authors	U at days	U at authors
Issue-type routing	92	90	50	82	0.051	0.056
PR-outcome audit	92	86	45	60	0.053	0.075
Backlog-attention routing	92	87	46	73	0.053	0.062
Core: `huggingface/datasets`	92	80	37	46	0.057	0.097
Core: `pandas-dev/pandas`	92	92	66	53	0.050	0.085
Core: `psf/requests`	92	90	66	79	0.051	0.058
Core: `streamlit/streamlit`	92	87	33	25	0.053	0.171

Table 18: Structural dispatch eligibility before and after admitting an exact read-only prologue (the API-documentation call these architectures open with), per released AppWorld baseline architecture; never pooled. The runtime rule and the measurement rule are the same function.

Released baseline architecture	Runs	Trajectories	Before (pos. 0)	Rate	After (pos. 0 or exact prologue)	Rate
Full code + reflection	8	2,340	2,339	1.000	2,339	1.000
Iterative parallel function calling	4	1,170	762	0.651	762	0.651
Plan and execute	8	2,340	1	0.000	1	0.000
ReAct	8	2,340	0	0.000	2	0.001

three. Table 17 reports the effective unit counts the live calibration cohorts actually contain; where a cluster-level count falls below 92 the corresponding conservative bound retires, which quantifies the exposure section 7 describes.

A read-only prologue does not recover ReAct dispatch. Section 5.3 shows eq. (2) refuses essentially every ReAct and plan-and-execute trajectory, which open with an API-documentation read. We built the narrowest relaxation (`paper/supplementary/read-only-prologue-protocol.md`): an artifact may declare an exact, allowlisted, read-only prologue whose calls must equal the committed observations in tool, order, and arguments; the prologue's results become live-ins of the region, its calls are not replayed, and any mismatch, extra call, order change, manifest mismatch, unknown effect, or slot left ungrounded by the extended entry contract falls back. The runtime passes the protocol's acceptance tests, including the archived suffix-dispatch pilot as a regression test. Measured on the 8,190 released trajectories with the documentation call as the prologue (table 18), it changes nothing: ReAct eligibility rises from 0 to 2 of 2,340 and plan-and-execute stays at 1. The looser diagnostic — any documentation-only prefix followed by the admitted program — recovers 2 and 3, and the program occurs anywhere in only 16 and 39 of those trajectories. Position was the proximate reason those architectures were refused; the underlying one is that they do not execute the admitted region at all, so no prologue rule can dispatch it. The prologue tool is also undeclared in the signed catalog, and the runtime refuses an undeclared prologue; the measurement assumes the declaration a reviewer would have to sign.

Why the workflow-reuse line is not a drop-in comparator. Section 7 states that we run no head-to-head against automated workflow reuse, plan caching, or call scheduling. The obstruction is that each optimizes a different quantity than the one our metric and our gate are defined over. Wang et al. (2025) induces reusable workflows into the *prompt*: the agent still reaches a model decision at every tool boundary, so the provider requests we count are not removed and there is no substituted region to guard. Kim et al. (2024) reschedules the calls of a single episode into a parallel DAG, which lowers latency without eliminating a model turn, and its planner remains in the loop. Zhang et al. (2025) reuses plans across episodes, which is the closest target, but without argument-level provenance or an effect declaration it makes no admissibility decision — so there is nothing to compare our gate against, only a cache-hit rate. Wei et al. (2026) compiles plans into an annotated intermediate representation and admits learned skills to a library only after verification tests, and Winston et al. (2026) compiles a task description into code validated against tool pre- and postconditions and schedules it for latency; in both, the model still plans each task, and admission is a test or specification check within an episode, not a bound on post-substitution risk over a calibration distribution, so neither supplies the admissibil-

Table 19: Recurrence-only replay on issue-type routing: per-record means over the 30 sealed held-out records. The first three rows are the retained arms; the last replays the three reads the compiler refused (`limit=100` has no witness) with every barrier disabled and one provider request. All four arms are 30/30 exact (McNemar $p = 1$ against each comparator); the replay is the cheapest arm, which is the price of the refusal on this cohort.

Arm	Exact	Requests	Tokens	Latency (s)	Cost (¢)
Unchanged agent	30/30	4.00	4,259	6.16	0.087
Compiled two-read prefix (GAC)	30/30	2.00	2,576	2.97	0.059
Hand-written bundle	30/30	2.00	1,781	3.25	0.055
Recurrence-only three-read replay	30/30	1.00	1,167	2.67	0.038

ity decision our gate would be compared against. Re-expressing any of them as a guarded pre-model substitution with an entry contract and a verifier would change the method being measured rather than benchmark it. The comparison we owe is internal. Table 20 prices the barriers: for each retained hazard it names the guard that caught it and what deterministic replay of the mined region would have done with provenance, effect, position, contract, and risk barriers disabled. On PR-outcome and backlog routing the mined three-read sequence *is* the admitted program, so an unguarded replay executes the same calls and differs only in guards that never fired on in-distribution held-out records; on issue-type routing the compiler refused the three-read candidate, so its price was measured live under the pre-registered protocol (`paper/supplementary/recurrence-only-ablation-protocol.md`). A barrier-free replay of the refused three-read region — no provenance check, guard, verifier, gate, or position check — handed to one provider request on the same sealed 30 records passed 30/30 exact contracts with one request per record (table 19); every excerpt was located in its source comment. By the decision rule fixed in advance, on this cohort the provenance refusal cost one request per record without a measured quality benefit, and the guard's value here is the refusal of an unwitnessed argument plus the retained issue-6602 counterexample, not a held-out failure. The measurement concerns one family, one cohort, and in-distribution records; it says nothing about behavior under shift, which the drift protocol below is written to test.

A second model family on the same cohorts. Every headline number comes from one model, so a pre-registered transfer (`paper/supplementary/second-model-replication-protocol.md`) reran the three primary families on `gpt-6-luna`, a different model generation at the same efficiency tier, keeping the sealed discovery traces, splits, and 90 held-out records and recompiling each artifact provider-free under a manifest that pins the new model; the recompiled artifacts are identical to the retained ones (same ids, programs, and 92/0 gates). The decision rule fixed in advance puts one finding first: on issue-type routing the compiled arm missed one record the baseline passed (#6532, an excerpt that pluralized one source word), so its preservation hypothesis fails there; a second record (#3859) failed in both arms through line-ending normalization, and backlog record 2648 failed in the baseline only. Every miss is an excerpt-fidelity error by the model on identical evidence, and the retained `gpt-5.6-luna` arms pass all three records. Structure transferred as predicted: requests per record were unchanged in every arm, the artifact dispatched on 30/30 records in each family, and pooled reductions were 66.6% requests, 63.0% tokens, 59.5% latency, and 60.1% cost against 66.6, 63.1, 64.2, and 58.7 on the calibrated model (table 22). We read the miss as the case the manifest pin exists for: the certificate is conditional on the pinned model, these artifacts were never calibrated on `gpt-6-luna`, and one compiled-only miss in 90 is what an uncertified transfer looks like. Re-discovery on the second model (design A, `paper/supplementary/second-model-rediscovery-protocol.md`) then ran the whole pipeline on `gpt-6-luna`'s own traces over the same sealed records. Issue-type routing reached 128/132 exact discovery traces and PR-outcome 132/132; the compiler admitted the identical artifacts (same ids and programs, 92/0 gates, one and two candidates at calibration as in the primary runs), which dispatched on 30/30 records and passed 30/30 in every arm, with reductions of 50.0/38.9/32.8 and 75.0/80.7/76.0 percent in requests, tokens, and cost; the transfer miss on record 6532 did not recur. Backlog-attention routing refused at compile time: 113 of its 132 discovery traces were exact, below the 116 the 16/8/92 split needs, because the model's own excerpts were ungrounded on 19 records; no held-out arm was run and no artifact is claimed. The paper therefore claims one cali-

Table 20: Mechanism-removal table: every retained hazard, the guard that caught it, and what a recurrence-only replay of the mined region would have done. Each row is backed by a retained result file in the artifact.

Hazard (where observed)	Guard that caught it	Recurrence-only replay	Retained source (`paper/results/`)
Ungrounded argument `issue_get_comments.limit=100`, recurring in 132/132 discovery traces (issue-type, held-out #4420 family)	provenance + synthesis (stage 1/4): `ungroundable_slot`	replays the literal; emits the three-read program	`github_natural_replication/results.json\code{compiler.candidates[0]}`
Clean tool replay, wrong downstream answer (Markdown URL dropped, issue 6602) (natural-order study, 17/18 → 18/18 after checked rendering)	continuation contract (post-model)	accepts the answer	`github_natural_live/continuation_replay.json`
Getter-named method that mutates state on 36/36 calls (BFCL executed gold plans)	signed effect declaration (stage 2)	compiles a write into the region	`external_benchmarks/bfcl_compiler_execution.json`
API-documentation prologue before the hot prefix (AppWorld released runs, 4,679/4,680 ReAct and plan-and-execute)	position invariant, Eq. (2), at dispatch	dispatches out of position	`external_benchmarks/appworld_dispatch_preflight.json`
Suffix region dispatched at entry: reordered and duplicated calls (archived pilot, 16.7% pass)	position invariant (compile-time `prefix_only`)	ships the fault	`github_live/pilot_2026-08-03/`
Result outside the induced hull (`pr.state=open`; 40/120 core records) (cross-repository core protocol)	induced verifier V; clean fallback	answers from an unseen class unchecked	`github_multirepo_pr_outcome_core/results.json\code{dispatch.reasons}`
Projection type mismatch (`merged_at`, record 7199; 13 events) (PR-outcome pilot)	composite projection verifier; fallback	emits a malformed evidence object	`pr_outcome/pilot_v1/failed_attempt_projection_2026-08-05.json`
Recurrent but under-supported families ($n_{\max}$ 26, 8, 15 < 92) (NESTFUL, API-Bank, BFCL)	stage-6 exact gate	deploys on recurrence alone	`nestful/results.json;api_bank_execution.json;bfcl_compiler_execution.json`
Skewed calibration pool (84/30/2 classes) (`pytorch/pytorch`, core protocol)	stage-6 exact gate ($U = 1.0$)	deploys	`github_multirepo_pr_outcome_core/results.json\code{repository_failures[0]}`

brated model family, reports a second on which the pipeline re-derived two of three artifacts and refused the third, and makes no cross-model certificate claim.

A second provider on the same cohorts. The same design ran on Anthropic's `claude-sonnet-5` through the official SDK, with the unchanged Agents-SDK harness behind a provider adapter (`paper/supplementary/second-provider-replication-protocol.md`): adaptive thinking at effort `low`, parallel tool calls disabled, and the same prompts, tools, structured answers, grader, and compiler settings. Discovery reached 126, 128, and 116 exact traces of 132. Issue-type and PR-outcome admitted the identical artifacts (same ids and programs, 92/0 gates, one and two candidates at calibration) and dispatched on 30/30 records; backlog met the split exactly and both candidates retired at calibration because only 91 and 90 groups were accepted at any threshold, leaving the zero-violation bound at 0.050 and 0.051, above α: the floor of fig. 4 binding on a second provider. The decision rule reports the compiled-only misses first: two on issue-type (records 4248 and 6829) and two on PR-outcome (5401 and 6988). Every miss on this provider, in every arm, is an excerpt that drops Markdown link markup or normalizes whitespace, the issue-6602 mechanism, and the baseline itself missed one and three records. Exact contracts were 29/28/28 of 30 on issue-type (one hand-written episode timed out twice and counts as a failure) and 27/27/29 on PR-outcome; requests fell 48.3% and 75.0%, tokens 40.4% and 84.9%, and cost 39.8% and 83.4% (table 21). The structure transfers across providers; excerpt fidelity is the model's, and the exact contract charges it to whichever arm produced it.

A compression comparator ran, and did not engage. Every headline resource number compares a compiled prefix against an *uncompressed* baseline, because deployed compression middleware reports token reductions of the same order on a comparable workload by an entirely different mechanism (Headroom Labs, 2026). A live Headroom v0.5.18 ablation adds `headroom_only/compiled_headroom` arms to the PR-outcome and backlog-attention families on the existing 30-record held-out cohort (`paper/supplementary/headroom-ablation-protocol.md`). Headroom attempted every eligible JSON tool result and pre-model evidence object and applied *none* of them, saving zero tokens in either family, with all four paired quality contrasts holding 30/30

Table 21: Second provider, design A: Anthropic `claude-sonnet-5` on the sealed cohorts through the unchanged pipeline. Reductions are against the same-provider baseline; the compiled-only misses are the finding the protocol reports first, and the backlog refusal is the 92-group floor binding.

	Exact held-out contract			Reduction (%)		
Family	Base	Compiled	Manual	Requests	Tokens	Cost
Issue-type routing	29/30	**28/30**	28/30	48.3	40.4	39.8
PR-outcome audit	27/30	**27/30**	29/30	75.0	84.9	83.4
Backlog-attention routing	RETIRE at calibration (91/92 groups, $U > \alpha$; 116/132 exact traces)					

Table 22: Same-cohort transfer to `gpt-6-luna`: the three primary families rerun with the sealed records, discovery traces, and recompiled artifacts behind table 1. Reductions are against the same-model baseline. The issue-type compiled-only miss (record 6532) is the finding the transfer protocol reports first; under re-discovery the compiler re-derives the identical issue-type and PR-outcome artifacts and refuses backlog routing on the second model's own traces.

	Exact held-out contract			Reduction (%)		
Family	Base	Compiled	Manual	Requests	Tokens	Cost
Transfer (design B): retained artifacts recompiled under the new pin						
Issue-type routing	29/30	**28/30**	30/30	50.0	39.2	33.0
PR-outcome audit	30/30	**30/30**	30/30	75.0	80.8	76.3
Backlog-attention routing	29/30	**30/30**	30/30	74.8	81.0	75.4
Weighted total ($n = 90$)	88/90	**88/90**	90/90	66.6	63.0	60.1
Re-discovery (design A): the pipeline run on the second model's own traces						
Issue-type routing	30/30	**30/30**	30/30	50.0	38.9	32.8
PR-outcome audit	30/30	**30/30**	30/30	75.0	80.7	76.0
Backlog-attention routing	RETIRE at compile time (113/132 exact discovery traces)					

at exact p=1. This is a null engagement at this payload shape, not a measured composition or a measured floor — it does not establish Headroom's behavior on larger contexts — but it does retire the narrower concern that our reduction is compression under another name: on this workload the two systems visibly do not touch the same bytes. It is a compression comparator only; it does not stand in for a workflow-reuse or unguarded-replay baseline. That rerun also re-executed the unchanged agent on the same 30 backlog records and scored 30/30, so the single baseline miss of table 1 did not recur.

A drift-parity protocol, pre-registered and not yet run. The strongest standing objection is that a hand-written program reaches the same 90/90 contracts as the compiled artifact and is cheaper on one family. A three-arm design isolates the *induced verifier* from the *program* by pairing the compiled artifact with a permissive no-op verifier and testing both against the hand-written program under the nine-family metamorphic perturbation suite already implemented in `evaluation/perturb.py` (reordering, duplication, formatting, empty/null fields, schema drift, tool errors and timeouts), asking whether the compiled artifact abstains where an unverified program would answer silently (`paper/supplementary/drift-robustness-ablation-protocol.md`). Status: pre-registered on the deterministic demonstration substrate, not executed; no result from it is reported here.

Where "unchanged baseline" is exact. Pre-emission rejection is exact on both runtime paths; only a staging-owning runner can restore byte-identical model input after an attempt has been emitted (appendix B). Without one, a post-commit verifier failure is an incident rather than a fallback, which is why the compiled region is confined to the pre-commit prefix.

How the calibration cohort must be sampled. The calibration cohort must be sampled before filtering on whether the baseline happened to exhibit the mined region, with violation labels defined for every dispatch-eligible member; post-selecting only recurrent successes would not satisfy Proposition 1. This is why the AppWorld substrate fixes its calibration share at exactly the 92 groups the bound requires and draws them by seeded shuffle over all clean tasks, rather than taking the tasks in which the mined family happens to have the deepest support.

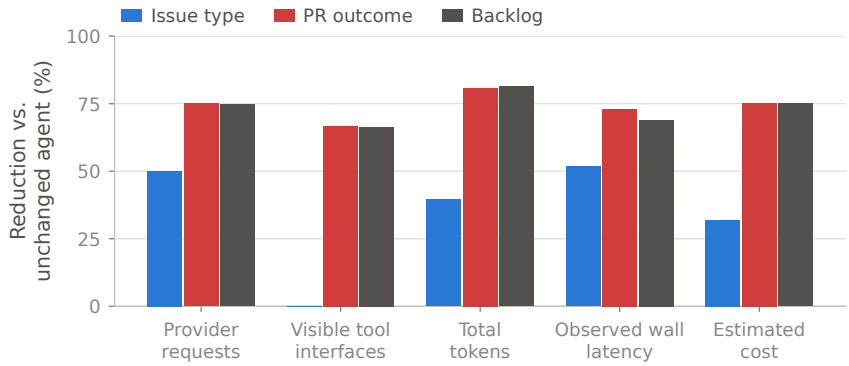


Figure 5: Per-family resource reductions behind table 1, plotted because the spread is the result: admissible prefix depth — not the optimizer — sets the savings, and the two deeper families sit at the feasibility ceiling of eq. (6).

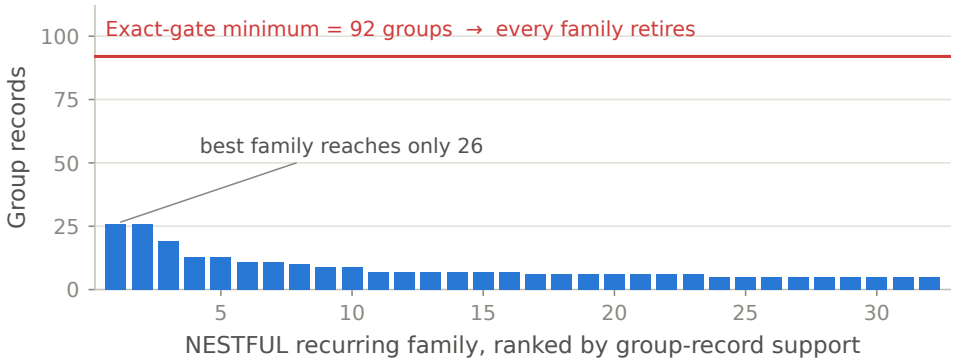


Figure 6: NESTFUL family supports against the 92-group requirement implied by eq. (7) at $\alpha = 0.05$, $\delta = 0.1$, $|\Lambda| = 11$. The largest family reaches 26 groups, so every family retires at calibration even though provenance, synthesis, and replay all succeeded.

Figure 5 plots the per-family spread behind table 1: admissible prefix depth, not the optimizer, sets the savings. The current empirical gate does not yet demonstrate a useful risk–coverage frontier at the registered 5% level. The strongest current runs are mostly all-or-none support tests (fig. 6), and the candidate-search problem is compiler-wide only when one candidate reaches calibration (Corollary 2) or the budget is divided across those that do; for the two families whose dominance search calibrated two candidates the corrected bound is 0.0569 (table 4). Likewise, exact task contracts do not certify full semantic equivalence of the final answer, especially once a downstream model continuation is involved. Two operational caveats of the reported runs have since been closed provider-free: the three primary artifacts were compiled with the perturbation suite disabled and registry signature verification off, and a recompilation from the sealed discovery checkpoints (`paper/scripts/recompile_with_challenge.py`) reproduces the identical programs, splits, artifact identifiers, and 92/0 gates with the suite enabled through a sandbox over the pinned snapshot; all nine perturbation families (reordering, duplicates, large and empty collections, nulls, schema drift, formatting, tool errors, timeouts) complete on the eight development windows of each artifact with zero wrong answers and zero hard rejects, so each record now carries `perturbations_claimed: true`. The same recompilation registers each artifact under an HMAC-SHA256 signing key, reloads it with verification on, and confirms that wrong-key, tampered, and the original unsigned registries are refused (`paper/results/iclr_revision/recompile_with_challenge.json`); no reported number changes.

H Prospective Gate-Frontier Protocol

`paper/supplementary/prospective-gate-frontier-protocol.md` pre-registers the experiment that could turn section 7's step-like gate into a genuine risk–coverage frontier. **Status: PRE-REGISTERED, THEN PARTIALLY EXECUTED.** The design below was committed before any repository or provider had been acquired under it, so it could not be revised in light of results it did not yet have; the cohort has since been sealed and executed on four of five repositories, reported at the end of this section without changing anything committed here.

Cohort. At least three repositories or domains, extending the provider-free `github_multirepo_preflight.py` harness, with at least 300 sealed held-out paired cases (zero compiled-only failures in 300 gives a one-sided 95% upper bound near 1%, well below the current small-cohort bounds), reported pooled and per-repository. **Compilation** freezes candidate selection throughout, so any admitted gate inherits Corollary 2 rather than a per-candidate certificate. **Three arms** compared under identical records, model, cache policy, and ordering: the unchanged agent, the learned gate of section 2, and a support-only frequency gate. **At least three distinct nonzero coverage levels** must be observed for the learned gate, which requires a cohort deliberately including record classes expected to sit at different points on a certification-difficulty gradient, not two endpoints from a homogeneous task. Thresholds, features, splits, hypotheses, and the stopping rule are fixed and hashed before acquisition.

Decision rule. A frontier with wrong-dispatch at or below the registered bound at every level, dominating the support-only gate at matched coverage, is the predicted positive outcome. A gate that stays step-like while the support-only comparator does not (or vice versa) is a reported negative for the learned score specifically. Any held-out wrong dispatch on either gate is an adverse finding reported in full, ahead of every other reading. Neither gate producing graded coverage is the fourth pre-declared outcome: the workload supplied no gradient, and no frontier is to be manufactured from a homogeneous cohort. Two extensions are named and explicitly deferred pending their own feasibility bar: model/provider breadth, and an executable workflow-compiler comparator (AWO is the closest published candidate), run only if it can match records, model, execution placement, overhead accounting, cache policy, and grading without hand-waving — a forced comparator would weaken the paper more than omitting one.

A pilot for this protocol has since run (`gate-frontier-pilot-protocol.md`, in the same directory): 90 records stratified by whether an issue's evidence comments contain a Markdown-style link, the one mechanism this codebase has recorded for a compiled artifact's excerpt to diverge from its source. Violations concentrated there ($4/60$ against $0/60$ at a matched baseline, one-sided $p = 0.059$) while a bare nearby URL predicted nothing ($1/60$, $p = 0.50$); one issue failed under both the unchanged agent and the compiled artifact independently. A weak go signal, not a frontier: it refines the calibration feature for the protocol above rather than substituting for it.

The comparator feasibility bar has also been checked (spike memo in the same directory): AWO (Abuzakuk et al., 2026) has no confirmed public implementation to build against, so this workstream closes on that no-go rather than on a forced, unverifiable reimplementation.

H.1 Observed results

The sealed cohort targets 300 pooled held-out pairs across the five repositories section 5.2 already uses, 116 discovery and 60 held-out cases each under frozen-candidate selection. Discovery reaches 580/580 exact traces. Four repositories admit an artifact; `pytorch/pytorch` again retires at compile time. This cohort is drawn from the same frozen snapshots with the same selection seed as the core study and excludes none of its records, so it shares 561/580 discovery, 442/460 calibration, and 130/150 held-out records with the core cohort; at twice the core scale it is a larger sealed re-execution, not an independent replication, and the `pytorch/pytorch` retirement is a near-replay of the core one on a 93–95% identical split. The pooled result is therefore 240 of the pre-registered 300 pairs, short of target for the stated reason.

Accounting. 240 records × 3 arms = 720 attempted episodes, 719 completed. Exact task contracts under intention-to-treat: unchanged agent 240/240, learned gate 240/240, support-only 239/240. The incomplete episode is `psf/requests` record 6708 under the support-only arm, which raised the 120 s provider timeout; the same record completed and passed in the other two arms. The protocol

Table 23: Disaggregated outcomes for the gate-frontier study. Exact contracts are reported per arm under intention-to-treat; † one support-only episode (record 6708) raised the 120 s provider timeout and was not re-attempted, while the record passed in the other two arms. "Dispatched" counts held-out records on which the compiled program ran; the remainder fell back to the unchanged agent on the guard clause `range:pr.state`. "Coverage in sweep" lists every calibration coverage the frozen grid produces for the learned gate. The support-only arm ($\alpha = 1$) reproduces the same n and k at every η but, because $\alpha = 1$ appends $\eta = 1.0$ to the grid, its Bonferroni split is $\delta/12$ ($U = 0.0507$ at $n = 92$) and it deploys threshold 1.0 rather than 0.5.

Repository	Held-out	Exact (B/L/S)	Dispatched	Coverage in sweep	Req. ↓
`huggingface/datasets`	60	60/60/60	40/60	0.1087 ($U = 0.375$, rejected), 1.0	44.4%
`pandas-dev/pandas`	60	60/60/60	40/60	1.0	44.4%
`psf/requests`	60	60/60/59†	40/60	1.0	44.4%
`pytorch/pytorch`	60 sealed	RETIRE	—	none ($U = 1$ at every η)	—
`streamlit/streamlit`	60	60/60/60	40/60	1.0	44.4%

fixed no retry rule, so the episode was not re-attempted and counts as a failure; among the 239 records with three completed arms every arm passes 239/239. Two provenance facts belong here. The `psf/requests` block was executed once, on 2026-08-22T18:59Z, as the single-repository pipeline check under the sealed selection; the five-repository run reused it through the driver's resume path rather than re-executing it. Retained spend for successful attempts sums to $0.41; the two failed `streamlit/streamlit` discovery attempts are not retained.

Dispatch. Calibration coverage is 1.0 in every admitting repository, but held-out dispatch is not: the compiled program ran on 160/240 records (learned gate) and 159/239 (support-only). The other 80 — every open-class pull request, 20 per repository — abstained on the induced verifier's `pr.state` hull, which the discovery pools (0–3 open records each) had fixed at {`closed`}, and ran the unchanged agent. Relative to baseline the learned gate reduces provider requests 44.4%, total tokens 52.3%, observed wall latency 51.0%, and estimated cost 48.3%; the support-only gate — $\alpha = 1$ in an otherwise identical second `compile_grc` pass — 44.4%, 52.4%, 47.9%, and 48.8%. Both arms admit the same artifact; they differ in α, in the grid ($\alpha = 1$ appends $\eta = 1.0$, so the split is $\delta/12$ and $U = 0.0507$ at $n = 92$), and in the deployed threshold (1.0 versus 0.5).

Coverage levels. Table 23 lists every coverage the frozen grid produces. Three admitting repositories accept no calibration group below $\eta = 0.11$ and all 92 at $\eta \geq 0.11$; `huggingface/datasets` additionally accepts 10/92 at $\eta \leq 0.08$, whose exact upper bound 0.375 exceeds α. No repository deploys at an intermediate coverage — and none could: with $|\mathcal{K}| = 92$ the registered bound certifies only $n_\eta = 92$, so the pre-registered criterion of three admissible nonzero coverage levels was unattainable at this cohort size whatever q ranks, and the protocol should have said so. Held-out q is moreover constant within each repository (0.0169; 0.0815), because the entry state is a single record number and every entry-observable feature is a function of it. Per the decision rule fixed above in this appendix, this is the fourth listed outcome: neither gate produces graded coverage. We report it as a null about the design, not as evidence about q: the study audits support-threshold behavior at full coverage under two risk budgets and does not test the ranking quality of q. The protocol that could (`paper/supplementary/graded-frontier-protocol.md`) needs $|\mathcal{K}| \geq \lceil 92/c \rceil$ groups per repository and an entry state with informative features; it is written and not run.

I Claims and Evidence

Tables 24 to 27 list every claim the paper makes, what was measured, the retained file that carries the number, and what the claim does not license. Rows are grouped as the paper is: the primary live families, the external substrates, and the studies added after the pre-submission review.

Table 24: Claims, evidence, retained sources, and claim boundaries, part A: the primary families. Every number is copied from the file named in the third column; the number registry in the artifact maps each to its producing script.

Claim	Evidence (what was measured)	Retained source	Claim boundary (what it does not license)
Preservation and reductions on the primary families	90/90 exact held-out contracts compiled, 89/90 unchanged agent, 90/90 manual program (three families, 30 records each). Pooled reductions: requests 66.6%, tokens 63.1%, latency 64.2%, cost 58.7%. Per family (requests/tokens/latency/cost): 50.0/39.5/51.7/32.0; 75.0/80.8/73.0/75.3; 74.8/81.4/68.9/75.1.	`paper/results/github_workflow_families/summary.json` (`overall`, `families[].reductions`); per-record arms in `paper/results/github_natural_replication/results.json` and `paper/results/github_workflow_families/pr_outcome/final/results.json`, `.../backlog_attention/final/results.json`.	Preservation on these 90 records, one snapshot, one calibrated model; not superiority, not equivalence, and not runtime dominance over the manual program (ties on requests, loses on tokens and cost, wins on latency).
Discordance	One discordant pair in 90: a baseline-only miss on backlog record 5189 (`comment_grounded` false) against zero compiled-only misses; exact McNemar $p = 1$; one-sided 95% upper bound on compiled-only discordance 3.3% pooled, 9.5% per family. The same 30 backlog records rerun on 2026-08-18 scored 30/30 in every arm.	`paper/results/iclr_revision/paired_statistics.json` (`discordance`); `paper/results/github_workflow_families/backlog_attention/headroom_ablation/results.json`.	A bound on this cohort's compiled-only discordance. A stochastic baseline miss is not evidence that the compiled arm is better.
Certificate level: per-candidate versus compiler-wide	Every primary artifact admitted at $n_\eta = 92$, $k_\eta = 0$, $U = 0.0498 \le 0.05$. Candidates reaching calibration: $m = 1$ (issue-type), $m = 2$ (PR-outcome, backlog). Under the corrected budget $\delta/(m\lvert\Lambda\rvert)$, $U = 0.0569$ for $m = 2$: those two are per-candidate at $\alpha = 0.05$ and compiler-wide only at $\alpha \ge 0.057$; 106 groups would be needed. Provider-free recompilation from the sealed checkpoints reproduces both admitted artifacts (ids, split digests, bounds).	`paper/results/iclr_revision/multiplicity_accounting.json` (`u_at_92`, `n_min`); `paper/results/iclr_revision/recompiled_candidates_pr_outcome.json`, `.../recompiled_candidates_backlog_attention.json`.	Conditional on i.i.d. calibration groups. The seven cross-repository artifacts froze one candidate before calibration and are compiler-wide at 0.0498.
The step-like gate is a sample-size floor	$U(n) = 1 - (\delta/\lvert\Lambda\rvert)^{1/n}$ reaches α at $n = 91.64$; $U(91) = 0.0503$, $U(92) = 0.0498$. Every calibration pool is $\lvert\mathcal{K}\rvert = 92$, so the certifiable coverage set is $\{0, 1\}$; selective coverage c needs $\lceil 92/c \rceil$ groups (115, 132, 184, 368 at $c = 0.8, 0.7, 0.5, 0.25$). All 7 registered artifacts are step-like (0 partial frontiers). The two observed intermediate coverages are uncertifiable: 10/92 at $U = 0.375$ (`huggingface/datasets`, gate frontier) and 90/92 at $U = 0.0509$ (`pandas-dev/pandas`, balanced rerun).	`src/guarded_agentic_compaction/grc/calibrate.py` (`clopper_pearson_upper`, $k = 0$); `paper/results/gate_selectivity_analysis.json` (`summary`); `paper/results/iclr_revision/gate_frontier_reanalysis.json` (`grid_rows`); `paper/iclr/figures/gate_floor.pdf`.	Does not test the ranking quality of q; two worlds (a score that cannot rank, a pool too small to certify a ranking) are not separated. The graded-frontier protocol is written and not run.
750 compiled episodes, one compiled-only failure	Compiled held-out episodes on the calibrated model: 90 (primary) + 120 (core) + 180 (balanced) + 240 (gate-frontier learned gate) + 120 (extended issue-type) = 750; 1 record where the baseline passed and the compiled arm failed (extended cohort, record 2737, an excerpt quoted with backticks absent from the source).	Per-record `results[]` of the five studies' `results.json` files; counted by `validate_live_extensions` in `paper/scripts/validate_artifacts.py`.	`gpt-5.6-luna` only, in-distribution records, one snapshot per study. Excludes the `gpt-6-luna` transfer, which produced one further compiled-only miss and is reported separately.

Table 25: Claims and evidence, part B: the cross-repository studies, the external substrates, and the recurrence-only replay.

Claim	Evidence (what was measured)	Retained source	Claim boundary (what it does not license)
Time-forward cross-repository core protocol	4/5 repositories admit: 580/580 exact discovery traces, 120/120 exact held-out pairs; the compiled program dispatched on 80/120 records (fallback `range:pr.state`, 10 per repository; discovery pools held 2/0/3/0 open records); reductions 44.4/52.4/49.4/48.6 (66.7% fewer requests on dispatched records). `pytorch/pytorch` retires at calibration: no group accepted at any threshold; discovery class mix 84 closed-unmerged / 30 open / 2 merged, 116 windows; the retirement is reproduced provider-free.	`paper/results/github_multirepo_pr_outcome_core/results.json`; `paper/results/iclr_revision/gate_frontier_reanalysis.json` (`core`); `paper/results/iclr_revision/pytorch_core_recompile.json`.	A two-read task; held-out records newer than discovery records. The retirement is the gate acting on a skewed discovery pool, not a model failure. A fixed two-read template is more efficient here.
Balanced three-repository rerun	All three PR classes in discovery: 360/360 discovery, 180/180 dispatched, 180/180 exact; reductions 66.7/78.4/60.7/72.7.	`paper/results/github_multirepo_pr_outcome_balanced/results.json`; `paper/results/github_multirepo_pr_outcome_balanced/repos/*/results.json`.	Shares 13, 25, and 8 of the 30-record core held-out sets (`paper/results/iclr_revision/cohort_overlap_audit.json`); not an independent replication of the core protocol. The two-read template ties.
Prospective gate-frontier study	5 repositories, 240 held-out triples: learned gate 240/240 exact, dispatched 160/240; support-only comparator ($\alpha = 1$) 239/239 with one timeout (record 6708); reductions 44.4/52.3/51.0/48.3 against 44.4/52.4/47.9/48.8; held-out q is constant within each repository (0.0169; 0.0815); retained spend $0.41.	`paper/results/github_multirepo_gate_frontier/results.json`; `paper/results/iclr_revision/gate_frontier_reanalysis.json` (`gate_frontier`).	A null about the design at $\lvert\mathcal{K}\rvert = 92$: the pre-registered three nonzero coverage levels were unattainable whatever q ranks; neither evidence for nor against q.
External substrates retire at the exact floor	NESTFUL 1,415 traces, $n_{\max} = 26$; API-Bank 212 traces, $n_{\max} = 8$; BFCL v4 executed gold plans 200 traces, 1,142/1,142 exact re-execution, $n_{\max} = 15$. Each is below 92, so every family retires; held-out replay pass/abstain/wrong is 24/12/0, 0/2/0, 3/3/0.	`paper/results/nestful/results.json` (`compiler.exact_gate`); `paper/results/external_benchmarks/api_bank_execution.json`; `paper/results/external_benchmarks/bfcl_compiler_execution.json`, `.../bfcl_gold_execution.json`.	No model ran; the refusals are settled arithmetically before compilation. No accuracy or efficiency claim; never pooled with the GitHub families.
AppWorld admits one artifact	136/147 gold solutions execute; 7,070/7,070 results reproduced byte for byte; $n_{\max} = 136 \geq 92$. One two-step artifact (`supervisor.show_profile` → `show_account_passwords`) admitted at 92/92 zero-violation groups, $U = 0.0498$, 11/11 exact on the sealed test split, under both pre-registered effect arms; four candidates retire ($U = 0.051$ at 89 and 90 eligible groups; $U = 1$ at 46 and 26).	`paper/results/external_benchmarks/appworld_compiler_execution.json` (`admission`, `exact_replay`, `effect_audit`); `paper/supplementary/appworld-compiler-protocol.md`.	Gold solutions, not agent trajectories; groups are tasks that share a scenario's supervisor and database; the admitted program is argument-free; no efficiency claim.
Admissible is not dispatchable	On 28 released baseline runs (8,190 trajectories) the admitted program is at position 0 in 3,102 (37.9%): full code + reflection 2,339/2,340, iterative parallel function calling 762/1,170, plan-and-execute 1/2,340, ReAct 0/2,340. 3,752 trajectories open with `api_docs.show_api_descriptions`; 61 contain the program elsewhere; 0.377 on `test_normal` against 0.380 on `test_challenge`.	`paper/results/external_benchmarks/appworld_dispatch_preflight.json` (`pooled`, `by_agent_family`, `first_call_in_released_trajectories`).	An upper bound on φ: the manifest check and calibrated gate are not evaluated; trajectories repeat tasks across runs; no request, token, or cost saving is measured.
Recurrence-only replay	The three-read region the compiler refused on issue-type routing, replayed with every barrier disabled and one provider request: 30/30 exact at 1.00 request, 1,167 tokens, 2.67 s, 0.038¢ per record; the retained arms are 30/30 at 4.00/2.00/2.00 requests.	`paper/results/recurrence_only_ablation/results.json` (`aggregate`, `per_field_exact`); `paper/supplementary/recurrence-only-ablation-protocol.md`.	One family, one cohort, in-distribution records: on this cohort the refusal bought no measured quality. Says nothing about shift or the other families.

Table 26: Claims and evidence, part C: the second model family, the second provider, the multiplicity-repair NO-GO, and group independence.

Claim	Evidence (what was measured)	Retained source	Claim boundary (what it does not license)
`gpt-6-luna` transfer (design B)	Retained artifacts recompiled under a manifest pinning `gpt-6-luna` (identical ids, programs, 92/0 gates); three families rerun on the sealed 90 records: compiled 88/90, baseline 88/90, manual 90/90. Issue-type 28/30 against 29/30 with one compiled-only miss (record 6532) and one both-arm miss (3859); backlog baseline-only miss 2648; 30/30 dispatch per family; reductions 66.6/63.0/59.5/60.1.	`paper/results/second_model_replication/summary.json`; `paper/results/second_model_replication/issue_type/results.json`; `paper/results/github_workflow_families/pr_outcome/gpt6_luna/results.json`, `.../backlog_attention/gpt6_luna/results.json`.	The artifacts were never calibrated on `gpt-6-luna`: an uncertified transfer, reported ahead of every other reading. No cross-model certificate.
`gpt-6-luna` re-discovery (design A)	The whole pipeline on the second model's own traces over the same sealed records. Issue-type: 128/132 exact discovery traces, $m = 1$, artifact `cand-01-1ebb8b2849c7` identical to the retained one, 92/0, $U = 0.0498$, 30/30 in every arm. PR-outcome: 132/132 exact traces, $m = 2$ (the two-read sibling dominated), artifact `cand-00-a1de3856bb6c` identical, 30/30 in every arm, 75.0% fewer requests. Backlog: 113/132 exact traces against the 116 required, so the compiler refuses at compile time and no held-out arm runs.	`paper/results/second_model_replication/issue_type_rediscovery/results.json`; `paper/results/github_workflow_families/pr_outcome/gpt6_luna_rediscovery/results.json`; `paper/results/github_workflow_families/backlog_attention/gpt6_luna_rediscovery/failure.json`; `paper/supplementary/second-model-rediscovery-protocol.md`.	Same snapshot and records. The backlog outcome is a principled refusal (decision-rule row 1), not a failure; each family's certificate follows m exactly as for the primary families.
Second provider: Anthropic `claude-sonnet-5` (design A)	The unchanged pipeline on Sonnet's own traces through an adapter over the official SDK. Issue-type: 126/132 exact traces, identical artifact ($m = 1$), 30/30 dispatch, exact 29/28/28 of 30 (one hand-written episode timed out twice). PR-outcome: 128/132, identical artifact ($m = 2$), 30/30 dispatch, 27/27/29. Backlog: 116/132 (the split's minimum); both candidates retire at calibration with 91 and 90 of 92 groups accepted ($U = 0.050, 0.051$). Compiled-only misses: 4248, 6829 (issue-type), 5401, 6988 (PR-outcome); every miss in every arm drops Markdown link markup or normalizes whitespace in the excerpt. Reductions 48.3/40.4/47.6/39.8 and 75.0/84.9/73.5/83.4.	`paper/results/second_provider_replication/issue_type/results.json`; `paper/results/github_workflow_families/pr_outcome/anthropic_sonnet5_rediscovery/results.json`; `.../backlog_attention/anthropic_sonnet5_rediscovery/failure.json`; `paper/supplementary/second-provider-replication-protocol.md`.	Same records and snapshot; not a cross-provider price comparison; the misses are the model's excerpt fidelity, charged to whichever arm produced them; the backlog refusal is the 92-group floor, not a trace-structure failure.
Multiplicity repair: NO-GO	The pre-registered fresh 106-group cohorts need 36 unused records per class. After excluding the 357 records used in any retained PR-outcome or backlog cohort, the snapshot holds 33 unused `open` pull requests and 8 unused `owned` issues, so the preflight recorded NO-GO; no provider call was made and the per-candidate wording stays.	`paper/results/multiplicity_repair/preflight.json`; `paper/supplementary/multiplicity-repair-protocol.md`.	Says nothing about the corrected bound itself. An unbalanced draw was considered and not taken; a later snapshot with the classes runs the design unchanged.
Group independence	The 92-record calibration cohorts span 86–90 distinct creation days, 60–82 authors, and 45–50 months per family; a bound that treats a day or an author as one unit retires every family.	`paper/results/iclr_revision/effective_units.json` (`primary_ranges`).	The binomial certificate is conditional on i.i.d. groups, which the snapshot does not establish.

Table 27: Claims and evidence, part D: the continuation check, the extended held-out cohort, the read-only prologue, and the closed operational caveats.

Claim	Evidence (what was measured)	Retained source	Claim boundary (what it does not license)
Continuation check (introduction example)	On 18 retained compiled cases, deterministic replay detection accepts 17 without repair; one case (issue 6602) carries a comment-evidence mismatch and checked rendering repairs it: 18/18 final contracts, 0 provider calls.	`paper/results/github_natural_live/continuation_replay.json` (`summary`).	Establishes detection and repair on retained outputs; no live latency, cost, or generalization claim.
Extended issue-type held-out	120 new records (40 bug / 40 enhancement / 40 other) on `gpt-5.6-luna` with the retained artifact recompiled under the current tracer pin (identical id, program, splits, 92/0 gate): baseline 118/120, compiled 118/120, manual 117/120; one compiled-only miss (record 2737), one baseline-only miss (3040), one three-arm miss (3968); dispatch 119/120 (record 5102 abstained on the verifier's cardinality clause); compiled-only bound 3.9% on 120 and 2.2% pooled over 210 records; reductions 50.0/39.5/42.5/32.8. All three pre-registered targets missed and reported; five records had appeared in the gate-frontier pilot cohort (none a missed record; 113/113/112 of 115 without them).	`paper/results/issue_type_extended_heldout/results.json` (`compiled_only_bound`, `dispatch`); `paper/supplementary/issue-type-extended-heldout-protocol.md`.	A different class mix from the primary cohort (`other` replaces `question`); Table 1 is unchanged. In-distribution, one snapshot, calibrated model.
Read-only prologue	The runtime admits an exact allowlisted read-only prologue (23 acceptance tests, including the archived suffix-dispatch pilot as a regression). Measured on the 8,190 released AppWorld trajectories with the documentation call as prologue: ReAct 0 → 2 of 2,340, plan-and-execute 1 → 1; the loosest documentation-prefix reading recovers 2 and 3; the program occurs anywhere in only 16 and 39.	`paper/results/external_benchmarks/appworld_dispatch_prologue_preflight.json` (`by_agent_family`); `tests/unit/test_dispatch_prologue.py`; `paper/supplementary/read-only-prologue-protocol.md`.	Position was the proximate refusal reason; those architectures do not execute the admitted region. The prologue tool is undeclared in the signed catalog; the measurement assumes the declaration a reviewer would sign.
Perturbation challenge and signed registries	Provider-free recompilation of the three primary artifacts with the perturbation suite enabled through a snapshot sandbox reproduces identical programs, splits, ids, and 92/0 gates; all nine families complete on the eight development windows with zero wrong answers and zero hard rejects (`perturbations_claimed: true`); HMAC-SHA256 signing with verification on refuses wrong-key, tampered, and the original unsigned registries.	`paper/results/iclr_revision/recompile_with_challenge.json`; `paper/scripts/recompile_with_challenge.py`.	Closes two disclosed operational caveats of the reported runs; changes no reported number.